Assessment and Answer Guide

DEVOPS ENGINEERING MASTERY — Volume 10, Companion Volume. First Edition, 9 November 2026. This guide contains 120 MCQs, 30 short-answer questions, 20 scenario-based troubleshooting questions, 10 incident case studies, the practical final examination and the capstone rubric — followed by a separate answer key with explanations. Attempt every question yourself before reading its answer; the explanation matters more than the letter.

> **WARNING:** Do not copy assignment solutions word for word. Read the outline, close this guide, and do the work yourself in your lab environment. That is where the learning happens.

Assessment and Answer Guide — Introduction

এই গাইডটি Volume 10-এর মূল্যায়ন সহচর। এতে আছে ১২০টি বহুনির্বাচনি প্রশ্ন (MCQ), ৩০টি সংক্ষিপ্ত প্রশ্ন, ২০টি ট্রাবলশুটিং সিনারিও, ১০টি প্রোডাকশন ইনসিডেন্ট কেস স্টাডি, একটি হাতে-কলমে ফাইনাল পরীক্ষা এবং সম্পূর্ণ উত্তর গাইড। প্রতিটি প্রশ্নের পাশে একটি difficulty label আছে — **Beginner**, **Intermediate** অথবা **Advanced**। Beginner মানে মৌলিক ধারণা, Intermediate মানে প্রয়োগ ও তুলনা, Advanced মানে প্রোডাকশন বিচার, ট্রেড-অফ ও ডিজাইন সিদ্ধান্ত।

কীভাবে ব্যবহার করবেন

- প্রথমে উত্তর না দেখে প্রশ্নগুলো নিজে সমাধান করুন। MCQ-র ক্ষেত্রে শুধু সঠিক অক্ষর নয়, কেন অন্য তিনটি বিকল্প ভুল — সেটিও নিজেকে বোঝান।
- Short-answer প্রশ্নের উত্তর ৩–৬ বাক্যে লিখুন; তারপর মডেল উত্তরের সঙ্গে তুলনা করুন। হুবহু মুখস্থ না করে যুক্তি মিলিয়ে নিন।
- Scenario ও Case Study-তে প্রথমে প্রমাণ (metrics, logs, traces) কী বলছে তা লিখুন, তারপর অনুমান, তারপর যাচাইয়ের ধাপ। অনুমানকে প্রমাণ বলে চালাবেন না।
- Final Exam ল্যাবে বসে, সময় ধরে করুন। পরীক্ষার আগে Answer Guide পড়বেন না — এতে নিজের প্রকৃত অবস্থা বোঝা যাবে না।
- Capstone Rubric দেখে নিজের কাজ নিজে মূল্যায়ন করুন; পাস মানে শুধু কাজ করা নয়, প্রমাণসহ ব্যাখ্যা করতে পারা।

প্রশ্নের সংখ্যা এক নজরে

অংশ	সংখ্যা	উদ্দেশ্য
MCQ Part 1 (1–60)	60	Observability ভিত্তি, Prometheus, PromQL, Grafana, Loki, Tracing, SLI/SLO

MCQ Part 2 (61–120)	60	Kubernetes, APM, Elastic/OpenSearch, Cloud, Retention, Security, Cardinality, HA
Short-Answer Questions	30	ধারণা নিজের ভাষায় ব্যাখ্যা
Troubleshooting Scenarios	20	বাস্তব সমস্যায় ধাপে ধাপে নির্ণয়
Production Incident Case Studies	10	প্রমাণ পড়ে root cause বিশ্লেষণ
Practical Final Examination	1	~3 ঘণ্টার হাতে-কলমে পরীক্ষা

সততার নোট

এই গাইডের উত্তর মুখস্থ করে পরীক্ষায় পাস করা যায়, কিন্তু প্রোডাকশন ইনসিডেন্টে তা কাজে আসবে না। সত্যিকারের দক্ষতা আসে ল্যাবে নিজে ভেঙে, মাপে, লগ পড়ে ও trace অনুসরণ করে। উত্তর দেখার আগে চেষ্টা করুন, ভুল হলে কারণ বুঝুন, তারপর আবার চেষ্টা করুন। কাল্পনিক কোম্পানি TechNova, ডোমেইন `technova.example` এবং IP `203.0.113.x` শুধু উদাহরণের জন্য; কোনো বাস্তব সিস্টেম, পাসওয়ার্ড বা গোপন তথ্য এখানে নেই।

Multiple-Choice Questions • Part 1 • Questions 1–60

Multiple-Choice Questions — Part 1 (1–60)

প্রতিটি প্রশ্নের একটি মাত্র সঠিক উত্তর আছে। Difficulty label দেখে নিজের স্তর যাচাই করুন।

1. [Beginner] Monitoring ও Observability-র মধ্যে মূল পার্থক্য কোনটি?

1. Monitoring জানা সমস্যা ধরে, observability অজানা সমস্যার কারণ খুঁজতে সাহায্য করে
2. Monitoring বিনামূল্যে, observability সবসময় ব্যয়বহুল
3. দুটি একই জিনিস, শুধু নাম আলাদা
4. Monitoring শুধু alert দেয়, observability শুধু dashboard দেয়

2. [Beginner] Observability-র তিনটি স্তম্ভ (three pillars) কোনগুলো?

1. CPU, Memory, Disk
2. Metrics, Logs, Traces
3. Alert, Dashboard, Report
4. Prometheus, Grafana, Kubernetes

3. [Beginner] Metrics বলতে সাধারণত কী বোঝায়?

1. ব্যবহারকারীর ব্যক্তিগত তথ্য
2. সার্ভারের পাসওয়ার্ড তালিকা
3. সময়ের সঙ্গে পরিবর্তিত সংখ্যাগত পরিমাপ, যেমন request সংখ্যা বা latency
4. অ্যাপ্লিকেশনের সোর্স কোড

4. [Beginner] Log-এর প্রধান কাজ কী?

1. সিস্টেমে ঘটে যাওয়া নির্দিষ্ট ঘটনার সময়সহ বিবরণ সংরক্ষণ করা
2. সার্ভারের গতি বাড়ানো
3. ডেটাবেস ব্যাকআপ নেওয়া
4. ব্যবহারকারীকে ইমেইল পাঠানো

5. [Beginner] Distributed trace কী দেখায়?

1. সার্ভারের তাপমাত্রা
2. একটি request বিভিন্ন সার্ভিসের মধ্য দিয়ে কীভাবে গেছে এবং কোথায় সময় ব্যয় হয়েছে
3. নেটওয়ার্ক কেবলের মানচিত্র
4. ডেটাবেসের সব টেবিলের তালিকা

6. [Beginner] Prometheus মূলত কী ধরনের ডেটা সংগ্রহ করে?

1. ব্যবহারকারীর স্ক্রিনশট
2. Log ফাইল

3. Trace span
4. Pull মডেলে time-series metrics

7. [Intermediate] Prometheus-এ Counter metric কখন ব্যবহার করবেন?

1. বর্তমান সক্রিয় সংযোগের সংখ্যা মাপতে
2. শুধু বাড়ে এমন মান, যেমন মোট request সংখ্যা, মাপতে
3. Latency-র শতকরা হার (percentile) মাপতে
4. সার্ভারের বর্তমান তাপমাত্রা মাপতে

8. [Intermediate] Gauge ও Counter-এর পার্থক্য কী?

1. Gauge শুধু বাড়ে, Counter বাড়ে-কমে
2. Gauge বাড়ে-কমে, Counter শুধু বাড়ে (reset ছাড়া)
3. দুটি একই, নাম আলাদা
4. Gauge শুধু string ধারণ করে

9. [Intermediate] Histogram কেন Summary-র চেয়ে Prometheus-এ বেশি পছন্দ করা হয়?

1. Histogram-এ bucket থাকে না
2. Histogram সার্ভারে percentile হিসাব করে, ক্লায়েন্টে নয়
3. Histogram দিয়ে সার্ভার-সাইডে যেকোনো percentile ও aggregation হিসাব করা যায়
4. Histogram কম জায়গা নেয় সবসময়

10. [Beginner] `up` metric-এর মান 0 হলে কী বোঝায়?

1. Target সুস্থ আছে
2. Prometheus target-টি scrape করতে পারেনি বা target down
3. ডিস্ক পূর্ণ হয়ে গেছে
4. Alertmanager বন্ধ আছে

11. [Intermediate] PromQL-এ `rate(http_requests_total[5m])` কী হিসাব করে?

1. গত 5 মিনিটে মোট request সংখ্যা
2. গত 5 মিনিটে প্রতি সেকেন্ডে গড় request হার
3. গত 5 মিনিটে সর্বোচ্চ latency
4. গত 5 মিনিটে error-এর শতাংশ

12. [Intermediate] `increase()` ও `rate()`-এর পার্থক্য কী?

1. rate() প্রতি সেকেন্ডে হার দেয়, increase() নির্দিষ্ট সময়ে মোট বৃদ্ধি দেয়
2. দুটি একই ফাংশন
3. increase() শুধু Gauge-এ কাজ করে
4. increase() হার দেয়, rate() মোট বৃদ্ধি দেয়

13. [Advanced] PromQL-এ `histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))` কেন `by (le)` দিয়ে aggregate করা জরুরি?

1. le ছাড়া query চলেই না
2. le লেবেল ছাড়া bucket গণনা করা যায় না
3. ভুল aggregation percentile-কে অর্থহীন করে; bucket boundary ধরে রাখতেই হয়
4. এটি শুধু query-কে দ্রুত করে

14. [Beginner] Exporter কী?

1. যে প্রোগ্রাম কোনো সিস্টেমের metrics Prometheus ফরম্যাটে প্রকাশ করে
2. Log ফাইল মুছে ফেলার টুল
3. Grafana dashboard export করার বোতাম
4. ডেটাবেস ব্যাকআপ টুল

15. [Beginner] Node Exporter কী মাপে?

1. শুধু অ্যাপ্লিকেশনের ব্যবসায়িক metrics
2. হোস্ট মেশিনের CPU, memory, disk, network metrics
3. শুধু Kubernetes Pod-এর সংখ্যা
4. ব্যবহারকারীর ব্রাউজার তথ্য

16. [Intermediate] Prometheus-এ scrape interval খুব ছোট (যেমন 1s) রাখলে কী ঝুঁকি?

1. Grafana ধীর হয় না
2. কোনো ঝুঁকি নেই, সবসময় ভালো
3. বেশি cardinality, storage ও target-এর উপর চাপ বাড়ে
4. Alert কখনো fire করে না

17. [Intermediate] Service discovery কেন ব্যবহার করা হয়?

1. নতুন Pod/service স্বয়ংক্রিয়ভাবে scrape target হিসেবে যুক্ত হয় বলে
2. Alertmanager বন্ধ করতে
3. Log সংরক্ষণ করতে
4. Target-এর IP হাতে লিখতে ভালো লাগে বলে

18. [Advanced] Prometheus federation কখন যুক্তিযুক্ত?

1. Log সংরক্ষণের জন্য
2. Alertmanager প্রতিস্থাপন করতে
3. প্রতিটি ছোট সার্ভিসের জন্য আলাদা Prometheus চালাতে
4. একাধিক Prometheus থেকে নির্বাচিত aggregate metrics একটি কেন্দ্রীয় Prometheus-এ আনতে

19. [Beginner] Alertmanager-এর প্রধান কাজ কী?

1. Alert গ্রহণ করে deduplicate, group, silence ও সঠিক receiver-এ পাঠানো
2. Dashboard আঁকা
3. Log parse করা
4. Metrics সংগ্রহ করা

20. [Intermediate] Alert rule-এ `for: 5m` দেওয়ার উদ্দেশ্য কী?

1. শর্ত 5 মিনিট ধরে সত্য থাকলেই alert fire করে, ক্ষণিকের spike এড়াতে
2. Alert 5 মিনিটে একবারই পাঠানো যায়
3. Prometheus 5 মিনিট বন্ধ থাকে
4. Alert 5 মিনিট পর মুছে যায়

21. [Intermediate] একই alert বারবার আসা (alert fatigue) কমানোর সবচেয়ে কার্যকর উপায় কোনটি?

1. শুধু ব্যবহারকারীর উপর প্রভাব ফেলে এমন, actionable alert রাখা এবং threshold ও for সময় ঠিক করা
2. Alert-এর সংখ্যা বাড়ানো
3. সবাইকে সব alert পাঠানো
4. সব alert বন্ধ করে দেওয়া

22. [Advanced] Alert-এ severity `critical` বনাম `warning` ঠিক করার সঠিক ভিত্তি কী?

1. কে alert তৈরি করেছে
2. যে metric-এর নাম বড়, সেটি critical
3. ব্যবহারকারীর উপর এখনই প্রভাব ও জরুরি মানব হস্তক্ষেপের প্রয়োজন আছে কি না
4. Alert-এর রং

23. [Beginner] Grafana মূলত কী করে?

1. সার্ভার চালু করে
2. কোড deploy করে
3. Metrics সংগ্রহ করে
4. বিভিন্ন data source থেকে ডেটা এনে dashboard-এ visualize করে ও alert করে

24. [Intermediate] Grafana dashboard-এ variable ব্যবহারের সুবিধা কী?

1. একটি dashboard দিয়ে একাধিক service, environment বা instance দেখা যায়
2. Dashboard দ্রুত মুছে যায়
3. Prometheus বন্ধ হয়ে যায়
4. Alert বন্ধ হয়ে যায়

25. [Intermediate] Grafana panel-এ "No data" দেখালে প্রথমে কী পরীক্ষা করবেন?

1. ব্রাউজার আনইনস্টল করা

2. Dashboard মুছে ফেলা
3. সার্ভার restart করা
4. Query-র time range, data source ও query-টি Prometheus-এ সরাসরি চালিয়ে দেখা

26. [Advanced] Grafana-তে একই dashboard-এ 50টি panel রাখা কেন খারাপ অভ্যাস?

1. Prometheus বন্ধ হয়ে যায়
2. দেখতে সুন্দর হয় না শুধু
3. প্রতিটি panel আলাদা query চালায়, লোড বাড়ে ও সিদ্ধান্ত নেওয়া কঠিন হয়
4. Grafana crash করে সবসময়

27. [Beginner] Loki কীসের জন্য ব্যবহৃত হয়?

1. Metrics সংরক্ষণ
2. Log সংগ্রহ ও query করা, Prometheus-এর মতো label-ভিত্তিক
3. Trace সংরক্ষণ
4. ডেটাবেস চালানো

28. [Intermediate] Loki ও Elasticsearch-এর মূল পার্থক্য কী?

1. Loki শুধু label index করে, পুরো log index করে না; তাই হালকা ও সস্তা
2. দুটি একইভাবে কাজ করে
3. Loki শুধু Windows-এ চলে
4. Loki পুরো log content index করে, Elasticsearch করে না

29. [Intermediate] LogQL-এ `{app="api"} |= "error"` কী করে?

1. api অ্যাপের সব log মুছে দেয়
2. api লেবেলের log-এর মধ্যে "error" শব্দ আছে এমন লাইন ফিল্টার করে
3. error লেবেল তৈরি করে
4. সব log-কে error বানায়

30. [Intermediate] Structured logging (JSON log) কেন unstructured log-এর চেয়ে ভালো?

1. পড়তে কঠিন বলে
2. level, trace_id, request_id-র মতো field আলাদা করে query ও filter করা যায়
3. জায়গা বেশি নেয় বলে
4. মানুষ পড়তে পারে না বলে

31. [Beginner] OpenTelemetry (OTel) কী?

1. Metrics, logs ও traces সংগ্রহের vendor-neutral standard ও টুলসেট
2. শুধু একটি dashboard
3. একটি ডেটাবেস

4. একটি ক্লাউড প্রোভাইডার

32. [Intermediate] OTel Collector-এ pipeline-এর তিনটি অংশ কী?

1. Alert, Dashboard, Log
2. Input, Output, Storage
3. Receiver, Processor, Exporter
4. Client, Server, Database

33. [Intermediate] Trace context propagation কেন জরুরি?

1. সার্ভিসগুলো যাতে একই trace ID দিয়ে span যুক্ত করতে পারে
2. নেটওয়ার্ক দ্রুত করতে
3. পাসওয়ার্ড শেয়ার করতে
4. Log মুছতে

34. [Advanced] Sampling-এ head sampling বনাম tail sampling-এর পার্থক্য কী?

1. Head sampling trace শেষ হওয়ার পর সিদ্ধান্ত নেয়, tail sampling শুরুতেই নেয়
2. Head sampling trace শুরুতেই রাখবে কি না ঠিক করে, tail sampling পুরো trace দেখে সিদ্ধান্ত নেয়
3. দুটি একই
4. Tail sampling শুধু metrics-এ হয়

35. [Beginner] Span কী?

1. একটি dashboard panel
2. একটি trace-এর মধ্যে একটি কাজের সময়কাল, যেমন একটি HTTP request বা DB query
3. সার্ভারের একটি ডিস্ক
4. একটি alert-এর নাম

36. [Intermediate] Jaeger ও Tempo-র মধ্যে পার্থক্য কী?

1. দুটি একই পণ্য
2. Tempo শুধু Windows-এ চলে
3. Jaeger শুধু metrics দেয়, Tempo শুধু logs দেয়
4. Jaeger trace visualize ও query করে, Tempo trace সংরক্ষণের ব্যাকএন্ড; দুটি একসঙ্গে ব্যবহার করা যায়

37. [Beginner] SLI কী?

1. একটি log level
2. সার্ভিসের নির্ভরযোগ্যতার পরিমাপযোগ্য নির্দেশক, যেমন সফল request-এর অনুপাত
3. একটি সার্ভারের নাম
4. একটি alert-এর ধরন

38. [Intermediate] SLO 99.9% মানে কী?

1. 1000 request-এর মধ্যে 999টি সফল হওয়ার লক্ষ্য
2. সার্ভার 99.9% সময় বন্ধ থাকবে
3. 99.9% log সংরক্ষণ করতে হবে
4. 99.9% alert পাঠাতে হবে

39. [Intermediate] Error budget কী?

1. Alert-এর সংখ্যা
2. Log-এর আকার
3. ভুল করার জন্য বরাদ্দ টাকা
4. 100% থেকে SLO বিয়োগ করলে যে অনুমোদিত ব্যর্থতার পরিমাণ থাকে

40. [Advanced] Error budget শেষ হয়ে গেলে সঠিক সিদ্ধান্ত কোনটি?

1. SLO মুছে ফেলা
2. Alert বন্ধ করে দেওয়া
3. নতুন ফিচার চালিয়ে যাওয়া
4. নতুন release থামিয়ে নির্ভরযোগ্যতার কাজে মনোযোগ দেওয়া

41. [Beginner] Golden Signals কোন চারটি?

1. Latency, Traffic, Errors, Saturation
2. Alert, Log, Trace, Metric
3. Up, Down, Warning, Critical
4. CPU, Memory, Disk, Network

42. [Intermediate] Latency মাপতে গড় (average) কেন বিভ্রান্তিকর হতে পারে?

1. কিছু ধীর request গড়ে ঢাকা পড়ে; p95/p99 দেখলে আসল অভিজ্ঞতা বোঝা যায়
2. গড় হিসাব করা যায় না
3. গড় শুধু Gauge-এ হয়
4. গড় সবসময় ভুল হয়

43. [Intermediate] Saturation বলতে কী বোঝায়?

1. কোনো resource কতটা পূর্ণ বা ব্যস্ত, যেমন CPU 95% বা queue ভর্তি
2. Log-এর সংখ্যা
3. Alert-এর সংখ্যা
4. সার্ভার বন্ধ হওয়া

44. [Advanced] Burn rate alert কেন শুধু error rate threshold-এর চেয়ে ভালো?

1. Burn rate শুধু CPU-তে কাজ করে

2. Burn rate রং সুন্দর করে
3. Burn rate বলে error budget কত দ্রুত শেষ হচ্ছে, তাই কতটা জরুরি তা বোঝা যায়
4. Burn rate alert কখনো fire করে না

45. [Beginner] Incident response-এ প্রথম কাজ কী?

1. প্রভাব কমানো (mitigate) ও যোগাযোগ শুরু করা, তারপর root cause খোঁজা
2. সব log মুছে ফেলা
3. সার্ভার বিক্রি করা
4. দোষী ব্যক্তিকে খোঁজা

46. [Intermediate] Blameless postmortem কেন গুরুত্বপূর্ণ?

1. রিপোর্ট লম্বা করতে
2. Incident ভুলে যেতে
3. কাউকে শাস্তি দিতে
4. মানুষ ভুল লুকায় না, সিস্টেমের দুর্বলতা খুঁজে সত্যিকারের শিক্ষা নেওয়া যায়

47. [Intermediate] Runbook কী?

1. একটি dashboard-এর নাম
2. একটি বইয়ের নাম
3. জানা সমস্যার জন্য ধাপে ধাপে লেখা নির্ণয় ও সমাধানের নির্দেশনা
4. একটি alert-এর রং

48. [Advanced] "Alert fires but no notification arrives" — প্রথমে কোথায় দেখবেন?

1. ব্যবহারকারীর ব্রাউজার
2. Grafana dashboard-এর রং
3. Alertmanager-এ alert পৌঁছেছে কি না, routing/receiver ও silence আছে কি না
4. সার্ভারের তাপমাত্রা

49. [Beginner] RED method (Rate, Errors, Duration) কাদের জন্য?

1. হোস্ট মেশিনের জন্য
2. Request-driven সার্ভিসের জন্য
3. শুধু ডেটাবেসের জন্য
4. শুধু নেটওয়ার্কের জন্য

50. [Intermediate] USE method (Utilization, Saturation, Errors) কাদের জন্য?

1. Request-driven সার্ভিসের জন্য
2. CPU, disk, network-এর মতো resource-এর জন্য
3. শুধু log-এর জন্য

4. শুধু trace-এর জন্য

51. [Intermediate] Prometheus-এ `sum by (service) (rate(http_requests_total[5m]))` কী করে?

1. Error হার দেখায়
2. সব service-এর মোট request হার একত্র করে
3. প্রতিটি service অনুযায়ী request হার যোগ করে আলাদা করে দেখায়
4. শুধু একটি service দেখায়

52. [Advanced] `topk(5, ...)` ব্যবহারের ঝুঁকি কী?

1. শুধু 5টি metric থাকে
2. কোনো ঝুঁকি নেই
3. সময়ের সঙ্গে কোন series top-এ আছে তা বদলে যায়, তাই graph হঠাৎ লাফায় ও তুলনা কঠিন হয়
4. Query চলেই না

53. [Beginner] Recording rule কেন ব্যবহার করা হয়?

1. Log রেকর্ড করতে
2. ভারী query-র ফলাফল আগে থেকে হিসাব করে নতুন metric হিসেবে সংরক্ষণ করতে
3. Alert মুছতে
4. Dashboard আঁকতে

54. [Intermediate] `promtool check rules` কী করে?

1. Rules ফাইলের syntax ও বৈধতা যাচাই করে
2. Dashboard তৈরি করে
3. Log মুছে দেয়
4. Prometheus চালু করে

55. [Intermediate] Grafana Alloy কী?

1. Metrics, logs ও traces সংগ্রহ করে Prometheus/Loki/Tempo-তে পাঠানোর agent
2. শুধু একটি dashboard theme
3. একটি ক্লাউড প্রোভাইডার
4. একটি ডেটাবেস

56. [Advanced] Log-এ trace_id রাখার প্রধান সুবিধা কী?

1. Log ছোট হয়
2. একই request-এর log, metric ও trace পরস্পরের সঙ্গে যুক্ত (correlate) করা যায়
3. Log দ্রুত মুছে যায়
4. পাসওয়ার্ড লুকানো যায়

57. [Beginner] Health check endpoint (`/health`) কী দেখায়?

1. সার্ভারের দাম
2. ডেটাবেসের সব তথ্য
3. অ্যাপ্লিকেশন চালু ও কাজ করার মতো অবস্থায় আছে কি না
4. ব্যবহারকারীর স্বাস্থ্য

58. [Intermediate] Blackbox exporter কী মাপে?

1. বাইরে থেকে endpoint-এর উপলব্ধতা, যেমন HTTP 200, TLS মেয়াদ, DNS
2. শুধু log
3. শুধু trace
4. সার্ভারের ভেতরের CPU

59. [Advanced] SLO-তে শুধু availability (uptime) মাপা কেন যথেষ্ট নয়?

1. সার্ভার চালু থাকলেও response ধীর বা ভুল হতে পারে; latency ও correctness-ও মাপতে হয়
2. Availability সবসময় 100% হয়
3. SLO-তে শুধু CPU মাপতে হয়
4. Availability মাপা যায় না

60. [Intermediate] Dashboard তৈরির আগে সবচেয়ে গুরুত্বপূর্ণ প্রশ্ন কোনটি?

1. কতগুলো panel হবে?
2. Grafana-র সংস্করণ কী?
3. কোন রং ব্যবহার করব?
4. এই dashboard দেখে কে, কোন সিদ্ধান্ত নেবে?

Multiple-Choice Questions • Part 2 • Questions 61–120

Multiple-Choice Questions — Part 2 (61–120)

Part 2-এ Kubernetes, APM, Elastic/OpenSearch, Cloud, retention, security, cardinality, sampling, high availability ও production operations নিয়ে প্রশ্ন আছে।

61. [Beginner] Kubernetes-এ Pod-এর CPU/memory ব্যবহার দেখতে কোন `kubectl` কমান্ড?

1. `kubectl logs`
2. `kubectl apply -f`
3. `kubectl get pods`
4. `kubectl top pod`

62. [Intermediate] kube-state-metrics কী দেয়?

1. Log ফাইল
2. Node-এর CPU তাপমাত্রা
3. Deployment, Pod, Service-এর অবস্থা (desired vs actual) সংক্রান্ত metrics
4. অ্যাপ্লিকেশনের ব্যবসায়িক metrics

63. [Intermediate] Kubernetes-এ Pod restart-এর সংখ্যা বাড়তে থাকলে প্রথমে কী দেখবেন?

1. ডেটাবেসের দাম
2. Pod-এর `kubectl logs` ও `describe`-এ events ও exit reason
3. শুধু Node-এর IP
4. Grafana-র theme

64. [Advanced] Prometheus-এ Kubernetes service discovery ব্যবহার করলে Pod IP বদলালেও scrape কেন চলতে থাকে?

1. Prometheus নিজে IP অনুমান করে
2. Alertmanager IP ঠিক করে দেয়
3. IP কখনো বদলায় না
4. API server থেকে Pod-এর বর্তমান তালিকা নিয়ে target আপডেট হয়

65. [Intermediate] Container-এর memory limit ছাড়িয়ে গেলে কী হয়?

1. Log মুছে যায়
2. কিছুই হয় না
3. OOMKilled হয়ে container restart হতে পারে
4. CPU বেড়ে যায় শুধু

66. [Beginner] APM (Application Performance Monitoring) মূলত কী করে?

1. শুধু log সংরক্ষণ করে
2. নেটওয়ার্ক কেবল পরীক্ষা করে
3. শুধু সার্ভার চালু করে
4. অ্যাপ্লিকেশনের ভেতরের ধীর function, DB query ও error চিহ্নিত করে

67. [Intermediate] Profiling (CPU/memory profile) কখন ব্যবহার করবেন?

1. Dashboard আঁকতে
2. Alert পাঠাতে
3. সার্ভার বন্ধ থাকলে
4. কোন function বা allocation বেশি CPU/memory খাচ্ছে তা খুঁজতে

68. [Intermediate] Elasticsearch কীসের জন্য ব্যবহৃত হয়?

1. শুধু metrics সংরক্ষণ
2. Log ও document search ও বিশ্লেষণ, পুরো text index করে
3. শুধু trace সংরক্ষণ
4. শুধু dashboard

69. [Advanced] Elasticsearch-এ mapping না বুঝে log পাঠালে কী সমস্যা হতে পারে?

1. Log ছোট হয়ে যায়
2. Dashboard সুন্দর হয়
3. কোনো সমস্যা হয় না
4. একই field কখনো string, কখনো number হিসেবে এলে mapping conflict হয় ও query ভেঙে যায়

70. [Intermediate] OpenSearch কী?

1. একটি Prometheus exporter
2. একটি ব্রাউজার
3. Elasticsearch-এর community-driven fork, search ও observability-র জন্য
4. একটি ক্লাউড প্রোভাইডার

71. [Beginner] Cloud observability-তে managed Prometheus/Grafana ব্যবহারের সুবিধা কী?

1. Alert কখনো লাগে না
2. Log লাগে না
3. নিজে storage, HA ও upgrade সামলাতে হয় না
4. খরচ সবসময় শূন্য হয়

72. [Advanced] Cloud-এ observability খরচ হঠাৎ বাড়লে প্রথমে কী পরীক্ষা করবেন?

1. ব্যবহারকারীর সংখ্যা

2. Dashboard-এর সংখ্যা
3. সার্ভারের রং
4. কোন metric/log/trace-এর volume ও cardinality বেড়েছে এবং retention কত

73. [Intermediate] Retention policy বলতে কী বোঝায়?

1. Alert কতদিন চলবে
2. Dashboard কতদিন থাকবে
3. কতদিন ডেটা রাখা হবে, তারপর মুছে ফেলা হবে
4. কতজন ব্যবহারকারী থাকবে

74. [Intermediate] Metrics downsampling কখন যুক্তিযুক্ত?

1. শুধু log-এ
2. সবসময়, সব metrics-এ
3. পুরোনো ডেটার জন্য কম resolution রাখলে storage বাঁচে, সাম্প্রতিক ডেটা বিস্তারিত থাকে
4. কখনোই নয়

75. [Advanced] High cardinality বলতে কী বোঝায়?

1. অনেকগুলো alert থাকা
2. অনেকগুলো dashboard থাকা
3. অনেকগুলো server থাকা
4. একটি metric-এ লেবেলের সম্ভাব্য মানের সংখ্যা বিশাল হওয়া, যেমন user_id বা request_id লেবেলে ব্যবহার

76. [Advanced] `user_id` লেবেল হিসেবে ব্যবহার করলে কী হয়?

1. Alert কমে যায়
2. কিছুই হয় না
3. প্রতিটি ব্যবহারকারীর জন্য আলাদা series তৈরি হয়ে cardinality বিস্ফোরণ ও storage/memory চাপ বাড়ে
4. Query দ্রুত হয়

77. [Intermediate] Cardinality কমানোর একটি উপায় কী?

1. আরও লেবেল যোগ করা
2. উচ্চ-cardinality লেবেল বাদ দেওয়া বা aggregate করা
3. Scrape interval 1s করা
4. Retention বাড়ানো

78. [Beginner] Log-এ PII (ব্যক্তিগত তথ্য) রাখা কেন বিপজ্জনক?

1. Log বড় হয় বলে
2. পাসওয়ার্ড, টোকেন বা ব্যক্তিগত তথ্য ফাঁস হলে নিরাপত্তা ও গোপনীয়তা ভাঙে
3. Log পড়তে কঠিন হয়

4. Dashboard ধীর হয়

79. [Intermediate] Log-এ পাসওয়ার্ড বা টোকেন না রাখার সঠিক উপায় কী?

1. পাসওয়ার্ড ছোট করা
2. Log-এ লিখে পরে মুছে ফেলা
3. Log করার আগেই redact/mask করা এবং কোড রিভিউতে যাচাই করা
4. Log বন্ধ করে দেওয়া

80. [Advanced] Observability ডেটায় access control কেন দরকার?

1. সবাই সবকিছু দেখলে কাজ দ্রুত হয়
2. Log/trace-এ সংবেদনশীল তথ্য থাকতে পারে; শুধু প্রয়োজনীয় ভূমিকা (role) অনুযায়ী দেখা উচিত
3. Access control খরচ কমায়
4. Dashboard সুন্দর হয়

81. [Intermediate] Prometheus high availability-র সহজ পদ্ধতি কোনটি?

1. একটি Prometheus চালানো
2. একই config দিয়ে দুটি Prometheus চালানো (replica), উভয়ে একই target scrape করে
3. Prometheus বন্ধ রাখা
4. শুধু Alertmanager চালানো

82. [Advanced] দুটি Prometheus replica থাকলে Alertmanager-এ duplicate alert কেন আসতে পারে?

1. Log duplicate করে
2. Alertmanager ভুল করে
3. দুটি Prometheus একই alert পাঠায়; Alertmanager deduplication ও grouping দিয়ে তা সামলায়
4. Grafana duplicate করে

83. [Intermediate] Thanos / Cortex / Mimir কিসের জন্য?

1. Prometheus-এর দীর্ঘমেয়াদি storage, HA ও global query-র জন্য
2. Dashboard আঁকার জন্য
3. Trace সংগ্রহের জন্য
4. Log parse করার জন্য

84. [Beginner] `kubectl logs -f pod-name` কী করে?

1. Pod-এর CPU দেখায়
2. Pod মুছে দেয়
3. Pod-এর log live দেখায়
4. Pod restart করে

85. [Intermediate] `kubectl describe pod`-এ কী পাওয়া যায়?

1. শুধু dashboard
2. শুধু Pod-এর IP
3. Pod-এর events, অবস্থা, restart কারণ, resource ও volume তথ্য
4. শুধু log

86. [Advanced] Kubernetes-এ `container_cpu_usage_seconds_total` দিয়ে CPU ব্যবহার বের করতে কোন PromQL ঠিক?

1. `rate(container_cpu_usage_seconds_total[5m])`
2. `increase(container_cpu_usage_seconds_total)`
3. `count(container_cpu_usage_seconds_total)`
4. `container_cpu_usage_seconds_total`

87. [Intermediate] Sampling 10% করলে কী হারানোর ঝুঁকি থাকে?

1. Metrics হারিয়ে যায়
2. Dashboard হারিয়ে যায়
3. কিছুই হারায় না
4. বিরল কিন্তু গুরুত্বপূর্ণ error trace হারিয়ে যেতে পারে

88. [Advanced] Tail sampling-এ error trace বেশি রাখার নিয়ম কেন যুক্তিযুক্ত?

1. সফল trace-এর চেয়ে error trace-ই তদন্তে বেশি কাজে আসে, তাই সেগুলো অগ্রাধিকার পায়
2. Error trace ছোট হয়
3. সফল trace মুছে যায়
4. Error trace সুন্দর হয় বলে

89. [Beginner] On-call rotation কেন থাকে?

1. Alert বাড়াতে
2. Dashboard দেখতে
3. সবাইকে একসঙ্গে জাগাতে
4. Incident-এর দায়িত্ব ভাগ করে ক্লান্তি কমাতে ও সবসময় কেউ দায়িত্বে থাকে তা নিশ্চিত করতে

90. [Intermediate] Alert-এ runbook link দেওয়া কেন ভালো?

1. On-call ইঞ্জিনিয়ার দ্রুত সঠিক ধাপ শুরু করতে পারে, অনুমানের সময় বাঁচে
2. Alert দ্রুত fire করে
3. Log কমে যায়
4. Alert সুন্দর দেখায়

91. [Intermediate] Synthetic monitoring কী?

1. Log তৈরি করা
2. Dashboard তৈরি করা
3. বাস্তব ব্যবহারকারীর তথ্য সংগ্রহ
4. কৃত্রিম request পাঠিয়ে বাইরে থেকে সার্ভিসের উপলব্ধতা ও latency পরীক্ষা করা

92. [Advanced] Real User Monitoring (RUM) ও Synthetic monitoring-এর পার্থক্য কী?

1. Synthetic শুধু log-এ চলে
2. RUM বাস্তব ব্যবহারকারীর অভিজ্ঞতা মাপে, Synthetic কৃত্রিম probe দিয়ে পরীক্ষা করে
3. দুটি একই
4. RUM শুধু সার্ভারে চলে

93. [Intermediate] SLO-তে `multi-window, multi-burn-rate` alert কেন ব্যবহার করা হয়?

1. Alert বেশি আসে বলে
2. দ্রুত burn ও ধীর burn আলাদা করে ধরা যায়, false positive কমে
3. Dashboard সুন্দর হয়
4. Log কমে যায়

94. [Beginner] `kubectl get events --sort-by=.lastTimestamp` কী দেখায়?

1. সব Service-এর IP
2. Dashboard
3. সব Pod-এর log
4. ক্লাস্টারের সাম্প্রতিক events, যেমন scheduling ব্যর্থতা বা crash

95. [Advanced] Prometheus-এ `absent()` ফাংশন কখন কাজে আসে?

1. Dashboard আঁকতে
2. Metric থাকলে alert করতে
3. কোনো metric একেবারেই আসছে না — যেমন job বন্ধ — তা ধরতে
4. Log খুঁজতে

96. [Intermediate] Log retention 30 দিন থেকে 7 দিনে কমালে কী হবে?

1. খরচ বাড়বে
2. Storage খরচ কমবে, কিন্তু 7 দিনের পুরোনো incident তদন্ত কঠিন হবে
3. Alert বাড়বে
4. Metrics হারিয়ে যাবে

97. [Intermediate] Grafana-তে data source provisioning YAML কেন ব্যবহার করা হয়?

1. Alert বন্ধ করতে

2. Log সংরক্ষণ করতে
3. Dashboard মুছতে
4. কোড হিসেবে data source সংজ্ঞায়িত করে পুনরুৎপাদনযোগ্য (reproducible) সেটআপ করা যায়

98. [Advanced] Observability platform নিজেই down হলে প্রথম ঝুঁকি কী?

1. ব্যবহারকারী খুশি হয়
2. Incident দেখা যায় না — blind spot তৈরি হয়; তাই platform-এর নিজের health আলাদাভাবে monitor করতে হয়
3. খরচ কমে যায়
4. Log বেড়ে যায়

99. [Intermediate] Change (deploy) ও incident-এর সম্পর্ক বুঝতে কোন correlation সবচেয়ে কাজে আসে?

1. Deploy সময়ের সঙ্গে error/latency বদলের সময় মিলিয়ে দেখা
2. Dashboard-এর রং
3. Log-এর আকার
4. Alert-এর সংখ্যা

100. [Advanced] একটি সার্ভিসের latency হঠাৎ বেড়েছে, কিন্তু CPU স্বাভাবিক — সবচেয়ে সম্ভাব্য দিক কোনটি?

1. Dashboard মুছে ফেলা
2. Alert বন্ধ করা
3. CPU বাড়ানো
4. Downstream সার্ভিস, DB query বা নেটওয়ার্ক অপেক্ষা (waiting) — trace ও DB metrics দেখা

101. [Intermediate] Database connection pool পূর্ণ হলে কী লক্ষণ দেখা যায়?

1. Dashboard দ্রুত হয়
2. CPU 0% হয়ে যায়
3. Request অপেক্ষা করে, latency বাড়ে ও timeout/error বাড়ে
4. Log মুছে যায়

102. [Advanced] `rate()`-এ range vector খুব ছোট (যেমন 1m) নিলে কী সমস্যা?

1. Query দ্রুত হয়
2. Alert কখনো fire করে না
3. কোনো সমস্যা নেই
4. Scrape interval-এর চেয়ে ছোট হলে ডেটা মিস হয় ও ফলাফল অস্থির হয়

103. [Beginner] Log level `ERROR`, `WARN`, `INFO`, `DEBUG`-এর মধ্যে কোনটি সবচেয়ে গুরুতর?

1. ERROR
2. DEBUG
3. INFO

4. WARN

104. [Intermediate] Production-এ DEBUG log সবসময় চালু রাখা কেন খারাপ?

1. Volume, খরচ ও noise বাড়ে; গুরুত্বপূর্ণ log খুঁজে পাওয়া কঠিন হয়
2. Alert বেড়ে যায়
3. Metrics হারিয়ে যায়
4. DEBUG log সুন্দর হয় না

105. [Advanced] Trace-এ span-এর parent-child সম্পর্ক ভেঙে গেলে (broken trace) সবচেয়ে সম্ভাব্য কারণ কী?

1. কোনো সার্ভিস trace context forward করেনি বা header হারিয়েছে
2. CPU বেশি
3. Log বেশি
4. Dashboard ভুল

106. [Intermediate] Grafana alert ও Prometheus alert rule-এর মধ্যে পার্থক্য কী?

1. Grafana alert শুধু log-এ হয়
2. Prometheus alert শুধু dashboard-এ হয়
3. দুটি একই
4. Prometheus alert PromQL-এ, Grafana alert একাধিক data source-এ query করে; দুটির lifecycle আলাদা

107. [Intermediate] Incident-এর সময় "communication" কেন technical fix-এর মতোই গুরুত্বপূর্ণ?

1. Alert কমে যায়
2. Log কমে যায়
3. রিপোর্ট লম্বা হয় বলে
4. ব্যবহারকারী ও টিম জানে কী হচ্ছে, duplicate কাজ ও আতঙ্ক কমে

108. [Advanced] একটি alert 2 AM-এ fire করেছে, কিন্তু ব্যবহারকারীর উপর কোনো প্রভাব নেই — সঠিক প্রতিক্রিয়া কী?

1. সার্ভার বন্ধ করা
2. তখনই সবাইকে জাগানো
3. পরদিন পর্যালোচনা করে threshold/severity ঠিক করা; non-actionable alert-কে paging থেকে সরানো
4. Alert মুছে ফেলা

109. [Beginner] `node_memory_MemAvailable_bytes` দিয়ে কী বোঝা যায়?

1. ব্যবহারের জন্য উপলব্ধ memory
2. CPU সংখ্যা
3. নেটওয়ার্ক গতি
4. মোট disk

110. [Intermediate] Disk usage 90% ছাড়ালে কেন আগেই alert দেওয়া উচিত?

1. Alert বেশি আসে বলে
2. Log বেড়ে যায় বলে
3. Disk সুন্দর দেখায় বলে
4. 100% হলে লেখা বন্ধ হয়ে সার্ভিস ভেঙে পড়ে; আগেই ব্যবস্থা নেওয়ার সময় থাকে

111. [Advanced] Prometheus-এ `label_replace` বা recording rule দিয়ে লেবেল বদলালে কী সতর্কতা দরকার?

1. কোনো সতর্কতা দরকার নেই
2. নতুন series তৈরি হয় ও cardinality বাড়তে পারে; উদ্দেশ্য স্পষ্ট থাকতে হবে
3. Query বন্ধ হয়ে যায়
4. Alert মুছে যায়

112. [Intermediate] OpenTelemetry-তে auto-instrumentation-এর সীমা কী?

1. Auto-instrumentation কাজই করে না
2. শুধু log দেয়
3. সবকিছুই ধরা যায়
4. সাধারণ HTTP/DB span পাওয়া যায়, কিন্তু ব্যবসায়িক যুক্তি (business logic)-র span নিজে যোগ করতে হয়

113. [Advanced] Microservices-এ একটি request ধীর — metrics স্বাভাবিক, log-এ কিছু নেই — পরবর্তী ধাপ কী?

1. Dashboard মুছে ফেলা
2. Alert বন্ধ করা
3. সার্ভার restart করা
4. Trace দেখে কোন span-এ সময় ব্যয় হচ্ছে তা খুঁজে বের করা

114. [Intermediate] SLO-র জন্য কোন metric সবচেয়ে ভালো — availability না latency?

1. শুধু availability
2. শুধু latency
3. ব্যবহারকারীর অভিজ্ঞতা অনুযায়ী দুটিই আলাদা SLI হিসেবে মাপা উচিত
4. কোনোটিই নয়

115. [Beginner] `kubectl top node` কী দেখায়?

1. Node-এর dashboard
2. Node-এর CPU ও memory ব্যবহার
3. Node-এর log
4. Node-এর IP ঠিকানা

116. [Advanced] Observability খরচ নিয়ন্ত্রণের সবচেয়ে কার্যকর কৌশল কোনটি?

1. Dashboard বাড়ানো

2. Alert বাড়ানো
3. সব ডেটা মুছে ফেলা
4. কী মাপতে হবে তা আগে ঠিক করা, cardinality নিয়ন্ত্রণ, sampling ও retention tier ঠিক করা

117. [Intermediate] Log aggregation-এ Alloy/Fluent Bit-এর কাজ কী?

1. Log মুছে ফেলা
2. বিভিন্ন উৎস থেকে log সংগ্রহ করে parse, filter ও Loki/Elasticsearch-এ পাঠানো
3. Dashboard আঁকা
4. Alert পাঠানো

118. [Advanced] একটি deploy-এর পর error rate বেড়েছে, কিন্তু পুরোনো সংস্করণে ফিরে গেলে (rollback) error কমে — এটি কী প্রমাণ করে?

1. Prometheus খারাপ
2. নেটওয়ার্ক খারাপ
3. সমস্যাটি নতুন change-এর সঙ্গে সম্পর্কিত হওয়ার শক্তিশালী প্রমাণ
4. ডেটাবেস খারাপ

119. [Intermediate] Dashboard-এ "single pane of glass" বলতে কী বোঝায়?

1. একটি alert
2. একটি জানালা
3. এক জায়গায় metrics, logs ও traces একসঙ্গে দেখে সিদ্ধান্ত নেওয়া
4. একটি panel

120. [Advanced] Production observability-র সবচেয়ে গুরুত্বপূর্ণ নীতি কোনটি?

1. সব alert critical করা
2. সব log DEBUG রাখা
3. যত বেশি ডেটা, তত ভালো
4. সঠিক ডেটা, সঠিক সময়ে, সঠিক মানুষের কাছে — যাতে সিদ্ধান্ত ও পদক্ষেপ নেওয়া যায়

Short-Answer Questions • 30 questions

Short-Answer Questions (30)

প্রতিটি প্রশ্নের উত্তর ৩–৬ বাক্যে নিজের ভাষায় লিখুন। সংজ্ঞার পাশাপাশি একটি উদাহরণ বা ট্রেড-অফ যোগ করুন।

1. **[Beginner]** Monitoring ও Observability-র পার্থক্য নিজের ভাষায় ব্যাখ্যা করুন।
2. **[Beginner]** Metrics, Logs ও Traces একসঙ্গে কীভাবে একটি incident বুঝতে সাহায্য করে?
3. **[Intermediate]** Counter, Gauge, Histogram ও Summary-র মধ্যে পার্থক্য ও ব্যবহারের উদাহরণ দিন।
4. **[Intermediate]** Prometheus কেন pull মডেল ব্যবহার করে? এর সুবিধা ও সীমা কী?
5. **[Intermediate]** PromQL-এ `rate()`, `increase()` ও `irate()`-এর পার্থক্য ব্যাখ্যা করুন।
6. **[Advanced]** Histogram-এ bucket নির্বাচন কেন গুরুত্বপূর্ণ? ভুল bucket কী সমস্যা করে?
7. **[Beginner]** Exporter কী এবং Node Exporter কী মাপে?
8. **[Intermediate]** Service discovery ছাড়া Kubernetes-এ Prometheus চালালে কী সমস্যা হয়?
9. **[Intermediate]** Alertmanager-এ grouping, deduplication ও silencing কীভাবে alert fatigue কমায়?
10. **[Advanced]** একটি ভালো alert-এর বৈশিষ্ট্য কী? "Actionable" বলতে কী বোঝায়?
11. **[Beginner]** Grafana dashboard ভালো করার তিনটি নীতি লিখুন।
12. **[Intermediate]** Grafana variable ও templating কীভাবে dashboard পুনঃব্যবহারযোগ্য করে?
13. **[Intermediate]** Loki কেন পুরো log index করে না? এর সুবিধা ও সীমা কী?
14. **[Intermediate]** LogQL-এ label filter ও line filter-এর পার্থক্য উদাহরণসহ ব্যাখ্যা করুন।
15. **[Beginner]** Structured logging কেন গুরুত্বপূর্ণ? একটি JSON log উদাহরণ দিন।
16. **[Intermediate]** OpenTelemetry Collector-এর receiver, processor ও exporter ব্যাখ্যা করুন।
17. **[Advanced]** Head sampling বনাম tail sampling — কখন কোনটি বেছে নেবেন?
18. **[Intermediate]** Trace context propagation ভেঙে গেলে কী হয় এবং কীভাবে ধরবেন?
19. **[Beginner]** SLI, SLO, SLA ও Error Budget-এর সম্পর্ক ব্যাখ্যা করুন।
20. **[Advanced]** Burn rate alert কী এবং এটি সাধারণ threshold alert-এর চেয়ে কেন ভালো?
21. **[Intermediate]** Golden Signals (Latency, Traffic, Errors, Saturation) দিয়ে একটি API কীভাবে মূল্যায়ন করবেন?
22. **[Intermediate]** RED ও USE method-এর পার্থক্য ও ব্যবহারের ক্ষেত্র লিখুন।
23. **[Beginner]** kube-state-metrics ও metrics-server-এর পার্থক্য কী?
24. **[Advanced]** High cardinality কী, কেন বিপজ্জনক, এবং কীভাবে কমাবেন?
25. **[Intermediate]** Log-এ PII ও secret ফাঁস রোধে কী কী ব্যবস্থা নেবেন?
26. **[Intermediate]** Retention ও downsampling-এর ট্রেড-অফ ব্যাখ্যা করুন।
27. **[Advanced]** Prometheus high availability কীভাবে করবেন? Replica ও long-term storage-র ভূমিকা কী?
28. **[Intermediate]** APM ও profiling কখন ব্যবহার করবেন? একটি উদাহরণ দিন।
29. **[Advanced]** Observability platform নিজেই down হলে কী করবেন? Blind spot এড়াতে কী ব্যবস্থা রাখবেন?
30. **[Intermediate]** Blameless postmortem কী এবং এটি কেন incident সংস্কৃতির জন্য জরুরি?

Troubleshooting Scenarios • 20 scenarios

Troubleshooting Scenarios (20)

প্রতিটি সিনারিও পড়ে: (ক) কী প্রমাণ আছে, (খ) সম্ভাব্য কারণগুলো কী, (গ) কোন ক্রমে যাচাই করবেন, (ঘ) সমাধান ও প্রতিরোধ — এই চারটি ধাপে উত্তর লিখুন।

1. **[Intermediate] Prometheus target shows down.** TechNova API-র target Prometheus-এ `down` দেখাচ্ছে, কিন্তু অ্যাপ্লিকেশন ব্রাউজারে খুলছে। কী কী কারণে এমন হতে পারে এবং কোন ক্রমে যাচাই করবেন?
2. **[Intermediate] Alert fires but no notification arrives.** Prometheus-এ alert `firing` অবস্থায় আছে, কিন্তু Slack/ইমেইলে কিছু আসছে না। কোথায় কোথায় দেখবেন?
3. **[Beginner] Grafana panel shows No data.** Dashboard-এর একটি panel-এ "No data" দেখাচ্ছে, অন্য panel ঠিক আছে। সম্ভাব্য কারণ ও যাচাইয়ের ধাপ লিখুন।
4. **[Advanced] High cardinality explosion.** Prometheus-এর memory হঠাৎ বেড়ে গেছে, active series 10 গুণ বেড়েছে। কীভাবে কারণ খুঁজবেন এবং তাৎক্ষণিক ও দীর্ঘমেয়াদি ব্যবস্থা কী?
5. **[Intermediate] Logs missing trace IDs.** Log-এ `trace_id` field খালি আসছে, তাই log থেকে trace-এ যাওয়া যাচ্ছে না। কোথায় ভাঙন হতে পারে?
6. **[Intermediate] Tempo shows broken traces.** Tempo-তে trace-এর কিছু span আলাদা আলাদা দেখাচ্ছে, parent-child সম্পর্ক নেই। কীভাবে নির্ণয় করবেন?
7. **[Beginner] Disk filling up on Prometheus server.** Prometheus সার্ভারের disk 92% পূর্ণ। তাৎক্ষণিক কী করবেন এবং দীর্ঘমেয়াদে retention কীভাবে ঠিক করবেন?
8. **[Advanced] Alert storm after a deploy.** একটি deploy-এর পর একসঙ্গে 40টি alert এসেছে। কীভাবে শান্তভাবে পরিস্থিতি সামলাবেন, কোন alert আগে দেখবেন?
9. **[Intermediate] p99 latency suddenly high, average normal.** গড় latency স্বাভাবিক, কিন্তু p99 হঠাৎ 3 সেকেন্ড। এর মানে কী এবং কোথায় খুঁজবেন?
10. **[Intermediate] Loki query is very slow.** Loki-তে 7 দিনের log query করতে 2 মিনিট লাগছে। কীভাবে query ও label নকশা ঠিক করবেন?
11. **[Advanced] Grafana dashboard loads slowly.** Dashboard খুলতে 30 সেকেন্ড লাগছে। Panel, query ও time range-এর দিক থেকে কী পরীক্ষা করবেন?
12. **[Intermediate] Node Exporter missing on new nodes.** Kubernetes-এ নতুন node যুক্ত হয়েছে, কিন্তু তার metrics আসছে না। Service discovery ও DaemonSet-এর দিক থেকে কী দেখবেন?
13. **[Advanced] False positive alerts at night.** প্রতি রাতে 2 AM-এ একটি alert fire করে, কিন্তু ব্যবহারকারীর উপর প্রভাব নেই। Threshold, for সময় ও severity কীভাবে ঠিক করবেন?
14. **[Intermediate] Logs stop arriving from one service.** একটি সার্ভিসের log হঠাৎ Loki-তে আসা বন্ধ হয়েছে, অন্য সার্ভিস ঠিক আছে। Alloy/agent থেকে Loki পর্যন্ত কোন ধাপে আটকে থাকতে পারে?
15. **[Advanced] Memory leak suspected in Node.js app.** অ্যাপের memory ধীরে ধীরে বাড়ছে, restart করলে কমে। কোন metrics, profile ও log দেখে leak নিশ্চিত করবেন?
16. **[Intermediate] 5xx errors spike after database restart.** DB restart-এর পর API-তে 5xx বেড়েছে। Connection pool, timeout ও retry-র দিক থেকে কী দেখবেন?

17. **[Beginner] Certificate expiry alert missed.** TLS certificate মেয়াদ শেষ হয়ে সাইট বন্ধ হয়েছে, কিন্তু কোনো alert আসেনি। Blackbox monitoring-এ কী অনুপস্থিত ছিল?
18. **[Advanced] Observability platform itself is down.** Grafana খুলছে না, Prometheus-ও সাড়া দিচ্ছে না। Incident তদন্তের জন্য কী বিকল্প উৎস ব্যবহার করবেন এবং blind spot কীভাবে এড়াবেন?
19. **[Intermediate] Sampling hides the error trace.** একটি গুরুত্বপূর্ণ error-এর trace খুঁজে পাওয়া যাচ্ছে না, সম্ভবত sampling-এ বাদ পড়েছে। কীভাবে নিশ্চিত হবেন ও sampling নীতি কীভাবে বদলাবেন?
20. **[Intermediate] Dashboard shows wrong numbers after a label change.** একটি deploy-এ লেবেলের নাম বদলে যাওয়ায় dashboard-এর সংখ্যা অর্ধেক দেখাচ্ছে। কী হয়েছে এবং কীভাবে ঠিক করবেন?

Production Incident Case Studies • 10 case studies

Production Incident Case Studies (10)

প্রতিটি কেসে background, timeline ও প্রমাণ (metrics/logs/traces) দেওয়া আছে। বিশ্লেষণ প্রশ্নগুলোর উত্তর প্রমাণের ভিত্তিতে লিখুন — অনুমানকে প্রমাণ বলে চালাবেন না।

Case Study 1 — CPU Saturation [Intermediate]

Background: TechNova API, শুক্রবার সন্ধ্যায় ট্রাফিক দ্বিগুণ। **Timeline:** 19:00 CPU 45%, 19:20 CPU 92%, 19:30 p95 latency 800ms থেকে 2.4s, 19:40 কিছু request timeout। **Evidence:** `node_cpu_seconds_total`-এ user CPU বেশি, `container_cpu_usage` একটি Pod-এ কেন্দ্রীভূত, log-এ timeout error, trace-এ API span-এই বেশি সময়।

Analysis questions: (ক) Saturation-এর প্রমাণ কী? (খ) কেন একটি Pod-এ চাপ কেন্দ্রীভূত হলো — load balancing না resource limit? (গ) তাৎক্ষণিক mitigation ও দীর্ঘমেয়াদি সমাধান কী?

Case Study 2 — Memory Leak [Advanced]

Background: Node.js API, প্রতি 3 দিনে Pod restart হয়। **Timeline:** Day 1 memory 300 MB, Day 2 700 MB, Day 3 1.4 GB ও OOMKilled। **Evidence:** `process_resident_memory_bytes` ধীরে ধীরে বাড়ছে, GC pause বাড়ছে, heap profile-এ একই object বাড়ছে, log-এ memory warning।

Analysis questions: (ক) Leak না স্বাভাবিক growth — কীভাবে পার্থক্য করবেন? (খ) কোন profiling ও log প্রমাণ leak নিশ্চিত করে? (গ) তাৎক্ষণিক ও স্থায়ী সমাধান কী?

Case Study 3 — Disk Exhaustion [Intermediate]

Background: Prometheus সার্ভার, disk 100% হয়ে Prometheus বন্ধ। **Timeline:** 7 দিন ধরে disk 70% → 85% → 100%, কোনো alert আসেনি। **Evidence:** `node_filesystem_avail_bytes` কমছে, Loki-তে log volume দ্বিগুণ, retention 30 দিন, alert threshold ছিল 95% কিন্তু `for: 1h` ছিল না।

Analysis questions: (ক) কেন alert আসেনি? (খ) Disk ভরার কারণ কীভাবে খুঁজবেন? (গ) প্রতিরোধে retention, alert ও cleanup-এ কী বদলাবেন?

Case Study 4 — API Latency Regression [Advanced]

Background: নতুন deploy-এর পর `/api/orders` ধীর। **Timeline:** Deploy 10:00, 10:05 p99 200ms → 1.8s, error বাড়েনি। **Evidence:** Trace-এ DB span 1.5s নিচ্ছে, DB-তে slow query log-এ full table scan, code-এ নতুন query-তে index নেই।

Analysis questions: (ক) Deploy-ই কারণ — এর প্রমাণ কী? (খ) কেন error বাড়েনি, শুধু latency বেড়েছে? (গ) Rollback না hotfix — কোনটি কখন?

Case Study 5 — App Crash Loop [Intermediate]

Background: নতুন Pod বারবার crash করছে, `CrashLoopBackOff`। **Timeline:** Pod চালু হয়, 10 সেকেন্ডে crash, আবার চালু হয়। **Evidence:** `kubectl describe`-এ exit code 1, log-এ "MongoDB connection refused", `kube_pod_container_status_restarts_total` বাড়ছে।

Analysis questions: (ক) Crash-এর root cause কী? (খ) কেন আগের সংস্করণ চলছিল? (গ) Readiness probe ও dependency check কীভাবে সাহায্য করত?

Case Study 6 — DB Connectivity Failure [Advanced]

Background: API থেকে DB-তে সংযোগ যাচ্ছে না, সব request 500 দিচ্ছে। **Timeline:** 14:00 DB restart, 14:02 API-তে 500 শুরু, 14:10 connection pool পূর্ণ। **Evidence:** Log-এ "connection timeout", DB-তে max connection পূর্ণ, trace-এ DB span-এ timeout, metrics-এ pool wait বাড়ছে।

Analysis questions: (ক) DB down, নাকি pool/config সমস্যা — কীভাবে পার্থক্য করবেন? (খ) Retry storm কেন পরিস্থিতি খারাপ করে? (গ) Circuit breaker ও pool sizing-এ কী শিখলেন?

Case Study 7 — 5xx Spike [Intermediate]

Background: হঠাৎ 5xx error 0.1% থেকে 12%। **Timeline:** 11:00 spike শুরু, 11:15 peak, 11:30 কমে যায়। **Evidence:** একটি downstream সার্ভিস 503 দিচ্ছে, তার নিজের CPU 100%, log-এ downstream timeout, trace-এ downstream span-এ error।

Analysis questions: (ক) নিজের সার্ভিসের দোষ, নাকি downstream-এর — প্রমাণ কী? (খ) Spike নিজে থেকে কমল কেন? (গ) Downstream failure-এ নিজের সার্ভিসকে কীভাবে রক্ষা করবেন?

Case Study 8 — Downstream Failure [Advanced]

Background: Payment gateway ধীর, checkout ব্যর্থ হচ্ছে। **Timeline:** Gateway latency 300ms → 5s, checkout success 98% → 60%। **Evidence:** Trace-এ gateway span-এই সময়, gateway-র status page-এ incident, নিজের API-র CPU/memory স্বাভাবিক।

Analysis questions: (ক) কোন প্রমাণ বলে সমস্যা নিজের সিস্টেমে নয়? (খ) ব্যবহারকারীকে কী জানাবেন ও কী fallback দেবেন? (গ) Timeout ও circuit breaker কীভাবে সেট করবেন?

Case Study 9 — Alert Storm [Intermediate]

Background: একটি network সমস্যায় একসঙ্গে 60টি alert। **Timeline:** 03:00 network flap, 03:01 60টি alert, on-call বিভ্রান্ত। **Evidence:** সব alert-এর একই root cause — একটি switch, কিন্তু প্রতিটি সার্ভিস আলাদা alert পাঠিয়েছে, grouping ছিল না।

Analysis questions: (ক) কেন 60টি alert এলো, যদিও কারণ একটিই? (খ) Grouping ও inhibition কীভাবে সাহায্য করত? (গ) পরের বার on-call কীভাবে অগ্রাধিকার ঠিক করবে?

Case Study 10 — Observability Platform Outage [Advanced]

Background: Grafana ও Prometheus দুটিই down, incident চলছে কিন্তু কিছু দেখা যাচ্ছে না। **Timeline:** 09:00 platform down, 09:20 API-তেও সমস্যা ধরা পড়ে ব্যবহারকারীর অভিযোগে। **Evidence:** Platform-এর নিজের health check ছিল না, log শুধু Loki-তে ছিল যা Grafana ছাড়া দেখা যায় না, backup dashboard ছিল না।

Analysis questions: (ক) Blind spot-এর কারণ কী? (খ) Platform down থাকাকালীন তদন্তের বিকল্প উৎস কী কী? (গ) Platform-এর নিজের monitoring ও redundancy কীভাবে নকশা করবেন?

Practical Final Examination • ~3 hours • Lab-based

Practical Final Examination

এই পরীক্ষা ল্যাবে বসে করতে হবে। নিজের Docker/Kubernetes ল্যাব, TechNova MERN অ্যাপ, Prometheus, Grafana, Loki ও Tempo ব্যবহার করুন। ইন্টারনেট থেকে উত্তর কপি করবেন না; নিজের ল্যাবের প্রমাণ (screenshot-এর বিবরণ, query ফলাফল, log লাইন) জমা দিন। মোট সময় ~3 ঘণ্টা।

Part A — Deploy the TechNova Observability Stack (45 মিনিট, 25 নম্বর)

- Docker Compose দিয়ে Prometheus, Grafana, Loki, Alloy ও Tempo চালু করুন। প্রতিটি সার্ভিসের health endpoint যাচাই করুন।
- Node Exporter ও TechNova API-কে Prometheus target হিসেবে যুক্ত করুন; `up` metric দিয়ে যাচাই করুন।
- Grafana-তে Prometheus, Loki ও Tempo data source যুক্ত করুন।

Part B — Instrument the MERN App (30 মিনিট, 20 নম্বর)

- TechNova API-তে `/metrics` endpoint যুক্ত করুন: `http_requests_total` (Counter), `http_request_duration_seconds` (Histogram)।
- Structured JSON logging যুক্ত করুন: প্রতিটি log-এ `level`, `trace_id`, `request_id`, `service` থাকতে হবে।
- OpenTelemetry auto-instrumentation চালু করে একটি request-এর trace Tempo-তে দেখান।

Part C — PromQL & LogQL (30 মিনিট, 20 নম্বর)

- PromQL লিখুন: (1) 5 মিনিটের error rate, (2) p95 latency, (3) প্রতি service-এ request হার, (4) CPU saturation।
- LogQL লিখুন: (1) গত 1 ঘণ্টায় `ERROR` log, (2) একটি `trace_id`-র সব log, (3) প্রতি মিনিটে error সংখ্যা (metric query)।
- প্রতিটি query-র ফলাফল কী বলছে, এক বাক্যে ব্যাখ্যা করুন।

Part D — Dashboards & Alerts (30 মিনিট, 20 নম্বর)

- Golden Signals অনুযায়ী একটি dashboard বানান: Latency, Traffic, Errors, Saturation — চারটি panel, প্রতিটির উদ্দেশ্য লিখুন।
- দুটি alert rule লিখুন: (1) 5xx error rate বেশি, (2) p95 latency SLO ভাঙছে। প্রতিটিতে `for`, severity ও runbook link থাকতে হবে।
- Alertmanager-এ একটি test alert পাঠিয়ে notification পৌঁছায় কি না যাচাই করুন।

Part E — Seeded Incident Investigation & Post-Incident Report (45 মিনিট, 15 নম্বর)

- পরীক্ষক একটি seeded incident চালু করবেন (যেমন ধীর DB query বা crash loop)। Metrics, logs ও traces দিয়ে root cause খুঁজুন।
- এক পৃষ্ঠার post-incident report লিখুন: timeline, প্রভাব, root cause, mitigation, প্রতিরোধমূলক পদক্ষেপ।

Time Guidance & Marking Scheme

Part	সময়	নম্বর
A — Deploy stack	45 মিনিট	25
B — Instrument app	30 মিনিট	20
C — PromQL & LogQL	30 মিনিট	20
D — Dashboards & Alerts	30 মিনিট	20
E — Incident & Report	45 মিনিট	15
মোট	**~3 ঘণ্টা**	**100**

পাস মান: মোট 60 নম্বর এবং Part E-তে অন্তত 8 নম্বর। শুধু config কপি করলে নম্বর মিলবে না — প্রতিটি ধাপের প্রমাণ ও ব্যাখ্যা থাকতে হবে।

Answer Guide • MCQs 1–60 • Short Answers • Scenarios

Answer Guide — MCQs 1–60, Short Answers, Scenarios

উত্তর দেখার আগে নিজে চেষ্টা করুন। প্রতিটি ব্যাখ্যায় কেন সঠিক উত্তরটি ঠিক এবং অন্য বিকল্পগুলো কেন ভুল — সেই যুক্তি দেওয়া হয়েছে।

MCQ 1–60 — Correct Answers with Explanations

1. A — Monitoring জানা সমস্যা ধরে, observability অজানা সমস্যার কারণ খুঁজতে সাহায্য করে। Monitoring আগে থেকে ঠিক করা threshold-এ alert দেয়। Observability metrics, logs ও traces দিয়ে এমন সমস্যার কারণ খোঁজা যায় যার কথা আগে ভাবা হয়নি।

2. B — Metrics, Logs, Traces। এই তিনটি স্তম্ভ একসঙ্গে সিস্টেমের অবস্থা, ঘটনা ও request-এর পথ দেখায়। CPU/Memory/Disk শুধু resource, স্তম্ভ নয়।

3. C — সময়ের সঙ্গে পরিবর্তিত সংখ্যাগত পরিমাপ। Metrics সংখ্যায় মাপা যায় ও সময়ের সঙ্গে তুলনা করা যায়। সোর্স কোড বা পাসওয়ার্ড metrics নয়।

4. A — নির্দিষ্ট ঘটনার সময়সহ বিবরণ সংরক্ষণ করা। Log বলে কখন কী ঘটেছে। এটি সার্ভারের গতি বাড়ায় না বা ব্যাকআপ নেয় না।

5. B — একটি request বিভিন্ন সার্ভিসের মধ্য দিয়ে কীভাবে গেছে। Trace span-এর মাধ্যমে দেখায় কোথায় সময় ব্যয় হয়েছে। এটি নেটওয়ার্ক মানচিত্র নয়।

6. D — Pull মডেলে time-series metrics। Prometheus target-এর `/metrics` endpoint থেকে ডেটা টেনে আনে। এটি log বা trace সংগ্রহ করে না।

7. B — শুধু বাড়ে এমন মান, যেমন মোট request সংখ্যা। Counter reset ছাড়া কখনো কমে না। সক্রিয় সংযোগের জন্য Gauge, percentile-এর জন্য Histogram দরকার।

8. B — Gauge বাড়ে-কমে, Counter শুধু বাড়ে। বর্তমান মান (যেমন সক্রিয় সংযোগ) Gauge দিয়ে মাপা হয়। Counter শুধু উপরে যায়, restart-এ শূন্য হয়।

9. C — Histogram দিয়ে সার্ভার-সাইডে যেকোনো percentile ও aggregation হিসাব করা যায়। Histogram bucket সংরক্ষণ করে, তাই পরে যেকোনো percentile বের করা যায়। Summary ক্লায়েন্টে percentile হিসাব করে, aggregate করা যায় না।

10. B — Prometheus target-টি scrape করতে পারেনি বা target down। `up==0` মানে scrape ব্যর্থ হয়েছে। এটি disk পূর্ণ হওয়ার প্রমাণ নয়।

11. B — গত 5 মিনিটে প্রতি সেকেন্ডে গড় request হার। `rate()` Counter-এর বৃদ্ধিকে প্রতি সেকেন্ডে হারে রূপান্তর করে। মোট সংখ্যার জন্য `increase()` ব্যবহার করুন।

12. A — rate() প্রতি সেকেন্ডে হার দেয়, increase() নির্দিষ্ট সময়ে মোট বৃদ্ধি দেয়। দুটি একই Counter থেকে আসে, কিন্তু একক আলাদা। increase() Gauge-এ কাজ করে না।

13. C — ভুল aggregation percentile-কে অর্থহীন করে; bucket boundary ধরে রাখতেই হয়। `histogram_quantile`-এর আগে `by (le)` দিয়ে bucket যোগ করতে হয়। le বাদ দিলে percentile ভুল হয়।

14. A — যে প্রোগ্রাম কোনো সিস্টেমের metrics Prometheus ফরম্যাটে প্রকাশ করে। Exporter সিস্টেমের ডেটাকে Prometheus পড়তে পারে এমন ফরম্যাটে রূপান্তর করে। এটি ব্যাকআপ টুল নয়।

15. B — হোস্ট মেশিনের CPU, memory, disk, network metrics। Node Exporter হোস্ট-স্তরের metrics দেয়। ব্যবসায়িক metrics অ্যাপ্লিকেশন নিজে দিতে হয়।

16. C — বেশি cardinality, storage ও target-এর উপর চাপ বাড়ে। খুব ছোট interval-এ বেশি ডেটা তৈরি হয় ও scrape চাপ বাড়ে। 15s বা 30s সাধারণত যথেষ্ট।

17. A — নতুন Pod/service স্বয়ংক্রিয়ভাবে scrape target হিসেবে যুক্ত হয়। Kubernetes-এ Pod IP বদলায়, তাই হাতে লেখা তালিকা টেকে না। Service discovery API থেকে বর্তমান তালিকা নেয়।

18. D — একাধিক Prometheus থেকে নির্বাচিত aggregate metrics একটি কেন্দ্রীয় Prometheus-এ আনতে। Federation global দৃশ্যের জন্য, সব raw ডেটা কপি করার জন্য নয়। Log সংরক্ষণ এর কাজ নয়।

19. A — Alert গ্রহণ করে deduplicate, group, silence ও সঠিক receiver-এ পাঠানো। Prometheus alert তৈরি করে, Alertmanager তা পরিচালনা করে। Dashboard আঁকা Grafana-র কাজ।

20. A — শর্ত 5 মিনিট ধরে সত্য থাকলেই alert fire করে, ক্ষণিকের spike এড়াতে। `for` ক্ষণিকের ওঠানামায় alert আটকায়। এটি alert মুছে ফেলার সময় নয়।

21. A — শুধু ব্যবহারকারীর উপর প্রভাব ফেলে এমন, actionable alert রাখা। অপ্রয়োজনীয় alert কমালে গুরুত্বপূর্ণ alert-এ মনোযোগ থাকে। সব alert বন্ধ করা বিপজ্জনক।

22. C — ব্যবহারকারীর উপর এখনই প্রভাব ও জরুরি মানব হস্তক্ষেপের প্রয়োজন আছে কি না। Critical মানে এখনই ব্যবস্থা নিতে হবে। Metric-এর নাম বা রং দিয়ে severity ঠিক হয় না।

23. D — বিভিন্ন data source থেকে ডেটা এনে dashboard-এ visualize করে ও alert করে। Grafana metrics সংগ্রহ করে না, Prometheus করে। Grafana দেখায় ও alert করে।

24. A — একটি dashboard দিয়ে একাধিক service, environment বা instance দেখা যায়। Variable বদলে একই dashboard পুনঃব্যবহার করা যায়। এটি Prometheus বন্ধ করে না।

25. D — Query-র time range, data source ও query-টি Prometheus-এ সরাসরি চালিয়ে দেখা। বেশিরভাগ "No data" ভুল query, ভুল time range বা ভুল data source থেকে আসে। প্রথমে query যাচাই করুন।

26. C — প্রতিটি panel আলাদা query চালায়, লোড বাড়ে ও সিদ্ধান্ত নেওয়া কঠিন হয়। অনেক panel মানে অনেক query ও বিভ্রান্তি। Dashboard সিদ্ধান্তের জন্য, সবকিছু দেখানোর জন্য নয়।

27. B — Log সংগ্রহ ও query করা, Prometheus-এর মতো label-ভিত্তিক। Loki log-এর জন্য, metrics-এর জন্য নয়। Trace-এর জন্য Tempo/Jaeger ব্যবহার করুন।

28. A — Loki শুধু label index করে, পুরো log index করে না; তাই হালকা ও সস্তা। Elasticsearch পুরো text index করে, তাই শক্তিশালী search দেয় কিন্তু বেশি resource লাগে। Loki query-র সময় log পড়ে।

29. B — api লেবেলের log-এর মধ্যে "error" শব্দ আছে এমন লাইন ফিল্টার করে। `{}` label filter, `|=` line filter। এটি log মুছে দেয় না।

30. B — level, trace_id, request_id-র মতো field আলাদা করে query ও filter করা যায়। JSON log-এ field আলাদা থাকে, তাই নির্ভুলভাবে খোঁজা যায়। Unstructured log-এ regex লাগে ও ভুল বেশি হয়।

MCQ 31–60 — Correct Answers with Explanations

31. A — Metrics, logs ও traces সংগ্রহের vendor-neutral standard ও টুলসেট। OpenTelemetry কোনো একক ভেন্ডরের পণ্য নয়। এটি instrumentation-এর মান ঠিক করে।

32. C — Receiver, Processor, Exporter। Receiver ডেটা নেয়, Processor রূপান্তর করে, Exporter পাঠায়। Pipeline-এর এই তিনটি অংশই মূল।

33. A — সার্ভিসগুলো যাতে একই trace ID দিয়ে span যুক্ত করতে পারে। Context header-এ পাঠানো না হলে trace ভেঙে যায়। এটি নেটওয়ার্ক দ্রুত করার উপায় নয়।

34. B — Head sampling trace শুরুতেই রাখবে কি না ঠিক করে, tail sampling পুরো trace দেখে সিদ্ধান্ত নেয়। Tail sampling error trace ধরে রাখতে পারে, কিন্তু বেশি resource লাগে। Head sampling সস্তা, কিন্তু গুরুত্বপূর্ণ trace হারাতে পারে।

35. B — একটি trace-এর মধ্যে একটি কাজের সময়কাল। প্রতিটি span একটি কাজের শুরু-শেষ ও সময় ধারণ করে। এটি disk বা alert নয়।

36. D — Jaeger trace visualize ও query করে, Tempo trace সংরক্ষণের ব্যাকএন্ড। দুটি একসঙ্গে ব্যবহার করা যায়: Tempo সংরক্ষণ করে, Jaeger/Grafana দেখায়। দুটি একই পণ্য নয়।

37. B — সার্ভিসের নির্ভরযোগ্যতার পরিমাপযোগ্য নির্দেশক। SLI যেমন সফল request-এর অনুপাত। এটি সার্ভারের নাম নয়।

38. A — 1000 request-এর মধ্যে 999টি সফল হওয়ার লক্ষ্য। 99.9% মানে 0.1% ব্যর্থতা অনুমোদিত। এটি সার্ভার বন্ধ থাকার হার নয়।

39. D — 100% থেকে SLO বিয়োগ করলে যে অনুমোদিত ব্যর্থতার পরিমাণ থাকে। 99.9% SLO-তে error budget 0.1%। বাজেট শেষ হলে release থামানো উচিত।

40. D — নতুন release থামিয়ে নির্ভরযোগ্যতার কাজে মনোযোগ দেওয়া। Error budget-এর উদ্দেশ্যই হলো ঝুঁকি ও গতির ভারসাম্য। SLO মুছে ফেলা সমস্যা লুকায়।

41. A — Latency, Traffic, Errors, Saturation। এই চারটি Golden Signals ব্যবহারকারীর অভিজ্ঞতা ও resource চাপ দেখায়। CPU/Memory/Disk শুধু resource।

42. A — কিছু ধীর request গড়ে ঢাকা পড়ে; p95/p99 দেখলে আসল অভিজ্ঞতা বোঝা যায়। গড় ভালো থাকলেও 5% ব্যবহারকারী ধীর অভিজ্ঞতা পেতে পারে। Percentile সেই অংশটি দেখায়।

43. A — কোনো resource কতটা পূর্ণ বা ব্যস্ত। Saturation মানে resource প্রায় পূর্ণ, যেমন CPU 95% বা queue ভর্তি। এটি সার্ভার বন্ধ হওয়া নয়।

44. C — Burn rate বলে error budget কত দ্রুত শেষ হচ্ছে। দ্রুত burn মানে জরুরি, ধীর burn মানে পরে দেখা যায়। শুধু error rate জরুরির মাত্রা বলে না।

45. A — প্রভাব কমানো (mitigate) ও যোগাযোগ শুরু করা, তারপর root cause খোঁজা। প্রথমে ব্যবহারকারীর ক্ষতি কমান, তারপর কারণ খুঁজুন। দোষ খোঁজা প্রথম কাজ নয়।

46. D — মানুষ ভুল লুকায় না, সিস্টেমের দুর্বলতা খুঁজে সত্যিকারের শিক্ষা নেওয়া যায়। দোষারোপ করলে মানুষ তথ্য লুকায়। Blameless সংস্কৃতিতে সিস্টেম ঠিক হয়।

47. C — জানা সমস্যার জন্য ধাপে ধাপে লেখা নির্ণয় ও সমাধানের নির্দেশনা। Runbook on-call-কে দ্রুত সঠিক ধাপে নিয়ে যায়। এটি বইয়ের নাম নয়।

48. C — Alertmanager-এ alert পৌঁছেছে কি না, routing/receiver ও silence আছে কি না। Prometheus-এ firing থাকলেও Alertmanager-এ routing ভুল, silence বা receiver ভুল থাকলে notification আসে না।

49. B — Request-driven সার্ভিসের জন্য। RED: Rate, Errors, Duration — API-র মতো সার্ভিসের জন্য। Resource-এর জন্য USE ব্যবহার করুন।

50. B — CPU, disk, network-এর মতো resource-এর জন্য। USE: Utilization, Saturation, Errors — resource মূল্যায়নের জন্য। Request-এর জন্য RED।

51. C — প্রতিটি service অনুযায়ী request হার যোগ করে আলাদা করে দেখায়। `by (service)` প্রতিটি service-এর জন্য আলাদা series রাখে। `by` ছাড়া সব একত্র হয়।

52. C — সময়ের সঙ্গে কোন series top-এ আছে তা বদলে যায়, তাই graph হঠাৎ লাফায়। topk-এ series বদলায়, তাই trend তুলনা কঠিন হয়। সতর্কভাবে ব্যবহার করুন।

53. B — ভারী query-র ফলাফল আগে থেকে হিসাব করে নতুন metric হিসেবে সংরক্ষণ করতে। Recording rule dashboard ও alert-কে দ্রুত করে। এটি log রেকর্ড করে না।

54. A — Rules ফাইলের syntax ও বৈধতা যাচাই করে। `promtool check rules` deploy-এর আগেই ভুল ধরে। এটি Prometheus চালু করে না।

55. A — Metrics, logs ও traces সংগ্রহ করে Prometheus/Loki/Tempo-তে পাঠানোর agent। Alloy একটি বহুমুখী telemetry agent। এটি ডেটাবেস নয়।

56. B — একই request-এর log, metric ও trace পরস্পরের সঙ্গে যুক্ত করা যায়। trace_id-ই তিনটি স্তম্ভের সংযোগ। Log ছোট করা এর উদ্দেশ্য নয়।

57. C — অ্যাপ্লিকেশন চালু ও কাজ করার মতো অবস্থায় আছে কি না। Health endpoint load balancer ও Kubernetes-কে জানায় অ্যাপ প্রস্তুত কি না। এটি ব্যবহারকারীর স্বাস্থ্য নয়।

58. A — বাইরে থেকে endpoint-এর উপলব্ধতা, যেমন HTTP 200, TLS মেয়াদ, DNS। Blackbox exporter বাইরে থেকে পরীক্ষা করে। ভেতরের CPU Node Exporter মাপে।

59. A — সার্ভার চালু থাকলেও response ধীর বা ভুল হতে পারে; latency ও correctness-ও মাপতে হয়। Availability 100% থাকলেও p99 5 সেকেন্ড হলে ব্যবহারকারী ক্ষতিগ্রস্ত হয়। তাই একাধিক SLI দরকার।

60. D — এই dashboard দেখে কে, কোন সিদ্ধান্ত নেবে? Dashboard-এর উদ্দেশ্য আগে ঠিক করুন, তারপর panel বাছুন। রং বা panel সংখ্যা পরে আসে।

Short Answers — Model Answers (30)

1. Monitoring আগে থেকে জানা সমস্যার জন্য threshold ও alert ব্যবহার করে, যেমন CPU 90% ছাড়ালে alert। Observability metrics, logs ও traces দিয়ে অজানা সমস্যার কারণ খুঁজতে দেয়। ভালো observability থাকলে "কেন ধীর?" প্রশ্নের উত্তর প্রমাণ দিয়ে দেওয়া যায়।

2. Metrics বলে কী ঘটছে (error rate বেড়েছে), log বলে কোন ঘটনা ঘটেছে (কোন request-এ error), trace বলে কোথায় সময় ব্যয় হয়েছে (কোন span ধীর)। একসঙ্গে তারা লক্ষণ থেকে কারণে পৌঁছাতে সাহায্য করে। একটিমাত্র স্তম্ভে পুরো ছবি পাওয়া যায় না।

3. Counter শুধু বাড়ে (মোট request), Gauge বাড়ে-কমে (সক্রিয় সংযোগ), Histogram bucket-এ latency বণ্টন ধরে (p95 বের করা যায়), Summary ক্লায়েন্টে percentile হিসাব করে। Histogram aggregate করা যায় বলে Prometheus-এ বেশি পছন্দ করা হয়। ভুল ধরন বাছলে query ভুল বা অসম্ভব হয়।

4. Pull মডেলে Prometheus নিজে target থেকে ডেটা টেনে আনে, তাই target-কে Prometheus-এর ঠিকানা জানতে হয় না। সুবিধা: service discovery সহজ, target down ধরা যায় (`up` metric)। সীমা: NAT/firewall-এর পেছনের target-এ পৌঁছানো কঠিন, সেজন্য Pushgateway বা remote write লাগতে পারে।

5. `rate()` প্রতি সেকেন্ডে হার দেয়, `increase()` নির্দিষ্ট সময়ে মোট বৃদ্ধি দেয়, `irate()` শেষ দুটি sample দিয়ে তাৎক্ষণিক হার দেয়। rate() স্থিতিশীল, irate() অস্থির কিন্তু দ্রুত পরিবর্তন ধরে। Alert-এ সাধারণত rate() ব্যবহার করা হয়।

6. Bucket হলো latency-র সীমা, যেমন 0.1s, 0.5s, 1s। Bucket খুব বড় হলে percentile মোটা হয়, খুব ছোট হলে অনেক series তৈরি হয়। সঠিক bucket অ্যাপের স্বাভাবিক latency-র চারপাশে থাকতে হয়। ভুল bucket-এ p95 অর্থহীন হয়।

7. Exporter কোনো সিস্টেমের metrics Prometheus ফরম্যাটে `/metrics` endpoint-এ প্রকাশ করে। Node Exporter হোস্টের CPU, memory, disk, network ও filesystem metrics দেয়। অ্যাপের নিজের metrics-এর জন্য client library দিয়ে instrumentation করতে হয়।

8. Service discovery ছাড়া প্রতিটি Pod-এর IP হাতে লিখতে হয়, কিন্তু Kubernetes-এ Pod IP বদলায়। নতুন Pod যুক্ত হলে বা পুরোনো Pod মরলে তালিকা পুরোনো হয়ে যায়। ফলে metrics হারায় ও target down দেখায়। Kubernetes service discovery API থেকে বর্তমান তালিকা নেয়।

9. Grouping একই ধরনের alert একসঙ্গে পাঠায়, deduplication একই alert বারবার পাঠানো আটকায়, silencing রক্ষণাবেক্ষণের সময় alert চুপ করায়। একসঙ্গে তারা alert-এর সংখ্যা কমিয়ে গুরুত্বপূর্ণ alert-এ মনোযোগ রাখে। ফলে on-call ক্লান্ত হয় না।

10. ভালো alert actionable হয় — অর্থাৎ alert পেয়ে কী করতে হবে তা স্পষ্ট থাকে, যেমন runbook link। এতে severity, প্রভাব ও `for` সময় ঠিক থাকে। Non-actionable alert শুধু noise তৈরি করে ও আসল alert উপেক্ষিত হয়।

11. প্রথমে ঠিক করুন কে দেখবে ও কোন সিদ্ধান্ত নেবে। Golden Signals অনুযায়ী panel সাজান। প্রতিটি panel-এর শিরোনাম ও একক স্পষ্ট রাখুন। অনেক panel না রেখে সিদ্ধান্তের জন্য প্রয়োজনীয় কয়েকটি panel রাখুন।

12. Variable (যেমন `$service`) দিয়ে একই dashboard বিভিন্ন service, environment বা instance-এর জন্য ব্যবহার করা যায়। Templating query-তে variable বসিয়ে panel পুনঃব্যবহারযোগ্য করে। ফলে প্রতিটি service-এর জন্য আলাদা dashboard বানাতে হয় না।

13. Loki শুধু label (যেমন app, level) index করে, পুরো log content index করে না। সুবিধা: কম storage ও খরচ, দ্রুত ingestion। সীমা: full-text search Elasticsearch-এর মতো শক্তিশালী নয়; query-র সময় log পড়তে হয়। তাই label নকশা গুরুত্বপূর্ণ।

14. Label filter `{app="api", level="error"}` কোন stream দেখবেন তা ঠিক করে। Line filter `|= "timeout"` সেই stream-এর মধ্যে নির্দিষ্ট শব্দ আছে এমন লাইন বেছে নেয়। Label filter আগে প্রয়োগ হয়, তাই তা সস্তা। ভুল label-এ query ধীর হয়।

15. Structured logging-এ প্রতিটি log JSON-এর মতো field-সহ লেখা হয়: `{"level":"error","trace_id":"abc123","msg":"DB timeout"}`। এতে level, trace_id বা request_id দিয়ে নির্ভুলভাবে filter করা যায়। Unstructured log-এ regex লাগে ও ভুল বেশি হয়।

16. Receiver OTel SDK বা অন্য উৎস থেকে telemetry গ্রহণ করে, Processor তাতে লেবেল যোগ, filter বা sampling করে, Exporter তা Prometheus, Loki বা Tempo-তে পাঠায়। Pipeline-এ এই তিনটি অংশ ক্রমানুসারে চলে। একাধিক pipeline দিয়ে metrics, logs ও traces আলাদাভাবে পরিচালনা করা যায়।

17. Head sampling trace শুরুতেই রাখবে কি না ঠিক করে — সস্তা, কিন্তু বিরল error হারাতে পারে। Tail sampling পুরো trace দেখে error বা ধীর trace রাখে — নির্ভুল, কিন্তু বেশি resource লাগে। সাধারণত 10% head sampling ও error-এর জন্য tail sampling একসঙ্গে ব্যবহার করা হয়।

18. Context propagation ভেঙে গেলে span-গুলো আলাদা trace হিসেবে দেখায়, parent-child সম্পর্ক হারায়। কারণ: কোনো সার্ভিস header forward করেনি, বা library-তে instrumentation নেই। নির্ণয়: প্রতিটি hop-এ traceparent header যাচ্ছে কি না পরীক্ষা করুন।

19. SLI হলো পরিমাপ (যেমন সফল request-এর অনুপাত), SLO হলো সেই পরিমাপের লক্ষ্য (99.9%), SLA হলো গ্রাহকের সঙ্গে চুক্তি, Error Budget হলো 100% থেকে SLO বিয়োগ করে পাওয়া অনুমোদিত ব্যর্থতা। বাজেট শেষ হলে release থামিয়ে নির্ভরযোগ্যতার কাজ করা হয়।

20. Burn rate বলে error budget কত দ্রুত শেষ হচ্ছে — যেমন 14.4x burn মানে বাজেট 2 ঘণ্টায় শেষ হবে। সাধারণ threshold শুধু বলে error বেশি, কিন্তু কতটা জরুরি তা বলে না। Multi-window burn rate দ্রুত ও ধীর burn আলাদা করে false positive কমায়।

21. Latency দিয়ে p95/p99 দেখুন, Traffic দিয়ে request হার, Errors দিয়ে 5xx অনুপাত, Saturation দিয়ে CPU/memory/queue চাপ দেখুন। চারটি একসঙ্গে দেখলে ব্যবহারকারীর অভিজ্ঞতা ও resource চাপ দুটিই বোঝা যায়। একটি signal একা যথেষ্ট নয়।

22. RED (Rate, Errors, Duration) request-driven সার্ভিসের জন্য, যেমন API। USE (Utilization, Saturation, Errors) CPU, disk, network-এর মতো resource-এর জন্য। API মূল্যায়নে RED, হোস্ট মূল্যায়নে USE ব্যবহার করুন। দুটি একসঙ্গে পুরো ছবি দেয়।

23. kube-state-metrics Kubernetes object-এর অবস্থা দেয় (যেমন Pod desired vs actual, restart সংখ্যা)। metrics-server শুধু বর্তমান CPU/memory ব্যবহার দেয় (`kubectl top` -এর জন্য)। একটি অবস্থা বোঝায়, অন্যটি resource ব্যবহার বোঝায়।

24. High cardinality মানে একটি metric-এ লেবেলের সম্ভাব্য মানের সংখ্যা বিশাল, যেমন user_id লেবেলে ব্যবহার। এতে লাখো series তৈরি হয়ে Prometheus-এর memory ও storage চাপ বাড়ে। সমাধান: উচ্চ-cardinality লেবেল বাদ দেওয়া, aggregate করা বা সীমিত করা।

25. Log-এ পাসওয়ার্ড, টোকেন, API key বা ব্যক্তিগত তথ্য লিখবেন না। Log করার আগেই redact/mask করুন, কোড রিভিউতে যাচাই করুন, Loki/Elasticsearch-এ access control দিন। ফাঁস হলে secret rotate করুন ও প্রভাব মূল্যায়ন করুন।

26. Retention বেশি রাখলে পুরোনো incident তদন্ত করা যায়, কিন্তু storage খরচ বাড়ে। Downsampling পুরোনো ডেটার resolution কমিয়ে খরচ বাঁচায়, কিন্তু বিস্তারিত হারায়। সাধারণত সাম্প্রতিক ডেটা বিস্তারিত ও পুরোনো ডেটা সংক্ষিপ্ত রাখা হয়।

27. একই config দিয়ে দুটি Prometheus replica চালান, উভয়ে একই target scrape করে। Alertmanager deduplication দিয়ে duplicate alert সামলায়। দীর্ঘমেয়াদি storage-র জন্য Thanos/Mimir-এর মতো সমাধান ব্যবহার করুন। এতে একটি Prometheus down হলেও monitoring চলে।

28. APM ব্যবহার করুন যখন কোন function, DB query বা downstream call ধীর তা জানতে হয়। Profiling ব্যবহার করুন যখন CPU বা memory কোন কোড খাচ্ছে তা জানতে হয়। উদাহরণ: p99 latency বেড়েছে, trace-এ DB span ধীর, profile-এ দেখা গেল একটি loop বেশি CPU খাচ্ছে।

29. Platform down হলে metrics, logs ও traces দেখা যায় না — blind spot তৈরি হয়। তাই platform-এর নিজের health আলাদাভাবে monitor করুন, backup dashboard ও `kubectl logs`-এর মতো বিকল্প রাখুন। Platform-এর জন্য আলাদা alert ও redundancy নকশা করুন।

30. Blameless postmortem-এ ব্যক্তিকে দোষ না দিয়ে সিস্টেমের দুর্বলতা খোঁজা হয়। এতে মানুষ ভুল লুকায় না, সত্য তথ্য আসে ও স্থায়ী সমাধান হয়। ফলে একই incident বারবার ঘটে না ও টিমের আস্থা বাড়ে।

Scenarios — Solution Reasoning (20)

1. Target down, অ্যাপ চলছে: প্রথমে Prometheus-এ target-এর error দেখুন (connection refused, timeout, 404)। তারপর `/metrics` endpoint সরাসরি curl করুন, firewall/port ও scrape config যাচাই করুন। অ্যাপ চললেও metrics endpoint বন্ধ, ভুল port বা network policy থাকতে পারে।

2. Alert firing, notification নেই: Alertmanager-এ alert পৌঁছেছে কি না দেখুন। Routing, receiver, silence ও inhibition যাচাই করুন। তারপর Slack/webhook credential ও নেটওয়ার্ক পরীক্ষা করুন। Prometheus-এ firing থাকা মানেই Alertmanager-এ পৌঁছেছে — এমন নয়।

3. No data: Time range, data source ও query যাচাই করুন। Query Prometheus-এ সরাসরি চালান। লেবেলের নাম বদলেছে কি না, metric আসছে কি না দেখুন। অন্য panel ঠিক থাকলে সমস্যা এই panel-এর query-তেই।

4. Cardinality explosion: `topk` দিয়ে কোন metric-এর series বেশি বেড়েছে তা খুঁজুন। নতুন deploy-এ নতুন লেবেল (যেমন user_id) যোগ হয়েছে কি না দেখুন। তাৎক্ষণিক: সমস্যার target-এর scrape বন্ধ বা relabel দিয়ে লেবেল বাদ দিন। দীর্ঘমেয়াদি: লেবেল নীতি ও cardinality alert ঠিক করুন।

5. Logs missing trace IDs: অ্যাপে logger-এ trace_id inject হচ্ছে কি না দেখুন। OTel context logger পর্যন্ত পৌঁছাচ্ছে কি না যাচাই করুন। Alloy-তে log parse করার সময় field হারাচ্ছে কি না দেখুন। একটি request-এর log ও trace পাশাপাশি মিলিয়ে পরীক্ষা করুন।

6. Broken traces: প্রতিটি সার্ভিস traceparent header forward করছে কি না দেখুন। কোনো hop-এ instrumentation নেই কি না খুঁজুন। Tempo-তে orphan span-এর service নাম দেখে ভাঙনের জায়গা চিহ্নিত করুন।

7. Disk 92%: তাৎক্ষণিক: পুরোনো WAL বা অপ্রয়োজনীয় ফাইল সরান, retention কমান। তারপর কোন ডেটা বেশি জায়গা নিচ্ছে তা দেখুন। দীর্ঘমেয়াদি: retention নীতি, disk alert (85% warning, 90% critical) ও capacity পরিকল্পনা ঠিক করুন।

8. Alert storm: প্রথমে সব alert না পড়ে grouping করুন — একই root cause-এর alert একসঙ্গে দেখুন। ব্যবহারকারীর উপর প্রভাব আছে এমন alert আগে দেখুন। Mitigation-এর পর alert নীতি পর্যালোচনা করুন।

9. p99 high, average normal: গড় ঢেকে রাখে, p99 বলে 1% request ধীর। Trace-এ ধীর request-এর span দেখুন — DB, downstream না GC? নির্দিষ্ট endpoint বা ব্যবহারকারীর ধরনে সমস্যা কি না দেখুন।

10. Loki slow query: Time range ছোট করুন, label filter নির্দিষ্ট করুন, `|=` line filter-এর আগে label filter ব্যবহার করুন। উচ্চ-cardinality লেবেল এড়ান। প্রয়োজনে query-কে ভাগ করে চালান।

11. Dashboard slow: Panel সংখ্যা কমান, ভারী query-কে recording rule দিয়ে প্রতিস্থাপন করুন, time range ছোট করুন। `$__rate_interval` ব্যবহার করুন। একই query বারবার চালানো panel খুঁজুন।

12. New node-এ Node Exporter নেই: DaemonSet সব node-এ Pod চালাচ্ছে কি না দেখুন। Service discovery-তে নতুন node আসছে কি না যাচাই করুন। Node-এর label বা taint Pod scheduling আটকাচ্ছে কি না দেখুন।

13. False positive at night: রাতের ট্রাফিক কম থাকলে ছোট সংখ্যায় error rate বেশি দেখায়। Threshold-এ ন্যূনতম traffic শর্ত যোগ করুন, `for` সময় বাড়ান, severity warning করুন। Non-actionable alert-কে paging থেকে সরান।

14. Logs stop from one service: Alloy-তে ওই সার্ভিসের log পৌঁছাচ্ছে কি না দেখুন। Service-এর log path, label ও Alloy config যাচাই করুন। Loki-তে ingestion error আছে কি না দেখুন। সার্ভিস restart হয়েছে কি না দেখুন।

15. Memory leak: `process_resident_memory_bytes` ধীরে বাড়ছে কি না, GC-র পরও কমছে কি না দেখুন। Heap profile-এ কোন object বাড়ছে তা খুঁজুন। Restart-এ কমে গেলে leak-এর সম্ভাবনা বেশি। কোডে cache বা event listener জমছে কি না দেখুন।

16. 5xx after DB restart: Connection pool-এর অবস্থা, DB-র max connection ও timeout দেখুন। Pool পূর্ণ হলে request অপেক্ষা করে ও timeout হয়। Retry storm পরিস্থিতি খারাপ করতে পারে। Pool size ও timeout ঠিক করুন।

17. Certificate expiry missed: Blackbox exporter-এ TLS expiry probe ছিল না। `probe_ssl_earliest_cert_expiry` দিয়ে 14 দিন আগে alert দিন। Certificate-এর মেয়াদ dashboard-এ দেখান।

18. Platform down: `kubectl logs`, `kubectl top` ও সরাসরি API curl দিয়ে তদন্ত করুন। ব্যবহারকারীর অভিযোগ ও status page ব্যবহার করুন। Platform-এর নিজের health check ও backup dashboard আগে থেকে রাখুন।

19. Sampling hides error trace: Sampling rate ও tail sampling নীতি দেখুন। Error trace-এর জন্য tail sampling-এ অগ্রাধিকার দিন। Log-এ trace_id থাকলে log থেকে trace খোঁজার চেষ্টা করুন।

20. Wrong numbers after label change: লেবেলের নাম বদলে গেলে পুরোনো ও নতুন series আলাদা হয়। Dashboard query-তে নতুন লেবেল ব্যবহার করুন, অথবা recording rule দিয়ে পুরোনো নামে map করুন। Deploy-এর আগে dashboard ও alert-এ লেবেলের প্রভাব যাচাই করুন।

Answer Guide • MCQs 61–120 • Case Studies • Final Exam • Capstone Rubric

Answer Guide — MCQs 61–120, Case Studies, Final Exam, Capstone Rubric

MCQ 61–90 — Correct Answers with Explanations

61. D — `kubectl top pod`। `kubectl top` metrics-server থেকে বর্তমান CPU/memory দেখায়। `get pods` শুধু অবস্থা দেখায়, log দেখায় না।

62. C — Deployment, Pod, Service-এর অবস্থা (desired vs actual) সংক্রান্ত metrics। kube-state-metrics Kubernetes API-র অবস্থাকে metrics-এ রূপান্তর করে। Node-এর তাপমাত্রা এর কাজ নয়।

63. B — Pod-এর `kubectl logs` ও `describe`-এ events ও exit reason। Restart-এর কারণ log ও event-এ থাকে। শুধু Node IP দেখে কারণ বোঝা যায় না।

64. D — API server থেকে Pod-এর বর্তমান তালিকা নিয়ে target আপডেট হয়। Service discovery Pod IP বদলালেও নতুন তালিকা নেয়। Prometheus IP অনুমান করে না।

65. C — OOMKilled হয়ে container restart হতে পারে। Memory limit ছাড়ালে kernel container-কে মেরে ফেলে। CPU বাড়া এর লক্ষণ নয়।

66. D — অ্যাপ্লিকেশনের ভেতরের ধীর function, DB query ও error চিহ্নিত করে। APM কোড-স্তরে কোথায় সময় ব্যয় হচ্ছে তা দেখায়। এটি শুধু log সংরক্ষণ করে না।

67. D — কোন function বা allocation বেশি CPU/memory খাচ্ছে তা খুঁজতে। Profile বলে কোন কোড বেশি resource খাচ্ছে। Dashboard আঁকতে profile লাগে না।

68. B — Log ও document search ও বিশ্লেষণ, পুরো text index করে। Elasticsearch full-text search-এ শক্তিশালী। এটি শুধু metrics সংরক্ষণ করে না।

69. D — একই field কখনো string, কখনো number হিসেবে এলে mapping conflict হয়। Elasticsearch field-এর ধরন একবার ঠিক হলে বদলানো কঠিন। Mapping আগে থেকে ঠিক করুন।

70. C — Elasticsearch-এর community-driven fork। OpenSearch search ও observability-র জন্য ব্যবহৃত হয়। এটি ক্লাউড প্রোভাইডার নয়।

71. C — নিজে storage, HA ও upgrade সামলাতে হয় না। Managed সার্ভিস পরিচালনার ভার কমায়। খরচ শূন্য হয় না, ব্যবহার অনুযায়ী বিল আসে।

72. D — কোন metric/log/trace-এর volume ও cardinality বেড়েছে এবং retention কত। খরচ বাড়ার কারণ সাধারণত volume বা retention। Dashboard সংখ্যা খরচ বাড়ায় না।

73. C — কতদিন ডেটা রাখা হবে, তারপর মুছে ফেলা হবে। Retention নীতি storage খরচ ও তদন্তের সময় নির্ধারণ করে। এটি ব্যবহারকারীর সংখ্যা নয়।

74. C — পুরোনো ডেটার জন্য কম resolution রাখলে storage বাঁচে। Downsampling সাম্প্রতিক ডেটা বিস্তারিত রেখে পুরোনো ডেটা সংক্ষিপ্ত করে। সব metrics-এ একই নিয়ম নয়।

75. D — একটি metric-এ লেবেলের সম্ভাব্য মানের সংখ্যা বিশাল হওয়া। user_id বা request_id লেবেলে ব্যবহার করলে লাখো series তৈরি হয়। অনেক server থাকা cardinality নয়।

76. C — প্রতিটি ব্যবহারকারীর জন্য আলাদা series তৈরি হয়ে cardinality বিস্ফোরণ হয়। user_id-র মান যত ব্যবহারকারী, তত series। এতে memory ও storage চাপ বাড়ে।

77. B — উচ্চ-cardinality লেবেল বাদ দেওয়া বা aggregate করা। লেবেল কমালে series কমে। আরও লেবেল যোগ করলে সমস্যা বাড়ে।

78. B — পাসওয়ার্ড, টোকেন বা ব্যক্তিগত তথ্য ফাঁস হলে নিরাপত্তা ও গোপনীয়তা ভাঙে। Log-এ PII থাকলে তা অনেক মানুষের কাছে পৌঁছে যায়। Log বড় হওয়া মূল সমস্যা নয়।

79. C — Log করার আগেই redact/mask করা এবং কোড রিভিউতে যাচাই করা। লেখার পর মুছে ফেলা যথেষ্ট নয়, কারণ log ইতিমধ্যে ছড়িয়ে যায়। Secret কখনো log-এ লিখবেন না।

80. B — Log/trace-এ সংবেদনশীল তথ্য থাকতে পারে; শুধু প্রয়োজনীয় ভূমিকা অনুযায়ী দেখা উচিত। Access control গোপনীয়তা রক্ষা করে। সবাই সবকিছু দেখলে ফাঁসের ঝুঁকি বাড়ে।

81. B — একই config দিয়ে দুটি Prometheus চালানো (replica)। দুটি replica একই target scrape করে, একটি down হলেও অন্যটি চলে। একটি Prometheus-এ HA হয় না।

82. C — দুটি Prometheus একই alert পাঠায়; Alertmanager deduplication দিয়ে তা সামলায়। Replica-র কারণে duplicate alert স্বাভাবিক। Alertmanager একই alert একত্র করে।

83. A — Prometheus-এর দীর্ঘমেয়াদি storage, HA ও global query-র জন্য। Thanos/Cortex/Mimir একক Prometheus-এর সীমা পেরিয়ে যায়। Dashboard আঁকা এদের কাজ নয়।

84. C — Pod-এর log live দেখায়। `-f` মানে follow, নতুন log আসতেই দেখায়। এটি Pod মুছে দেয় না।

85. C — Pod-এর events, অবস্থা, restart কারণ, resource ও volume তথ্য। describe Pod-এর পুরো অবস্থা দেখায়। শুধু IP বা log নয়।

86. A — `rate(container_cpu_usage_seconds_total[5m])`। CPU usage Counter, তাই হার বের করতে rate() লাগে। শুধু Counter-এর মান দিয়ে ব্যবহার বোঝা যায় না।

87. D — বিরল কিন্তু গুরুত্বপূর্ণ error trace হারিয়ে যেতে পারে। 10% sampling-এ 90% trace বাদ যায়। বিরল error-এর trace হারানোর ঝুঁকি থাকে।

88. A — সফল trace-এর চেয়ে error trace-ই তদন্তে বেশি কাজে আসে। Tail sampling error trace অগ্রাধিকার দিয়ে রাখে। সফল trace কম রাখলেও ক্ষতি কম।

89. D — Incident-এর দায়িত্ব ভাগ করে ক্লান্তি কমাতে ও সবসময় কেউ দায়িত্বে থাকে তা নিশ্চিত করতে। Rotation ছাড়া একই ব্যক্তি সবসময় জেগে থাকলে ভুল বাড়ে। এটি alert বাড়ানোর উপায় নয়।

90. A — On-call ইঞ্জিনিয়ার দ্রুত সঠিক ধাপ শুরু করতে পারে। Runbook link থাকলে অনুমানের সময় বাঁচে। Alert সুন্দর দেখানো এর উদ্দেশ্য নয়।

MCQ 91–120 — Correct Answers with Explanations

91. D — কৃত্রিম request পাঠিয়ে বাইরে থেকে সার্ভিসের উপলব্ধতা ও latency পরীক্ষা করা। Synthetic monitoring বাস্তব ব্যবহারকারী ছাড়াই পরীক্ষা করে। RUM বাস্তব ব্যবহারকারীর ডেটা নেয়।

92. B — RUM বাস্তব ব্যবহারকারীর অভিজ্ঞতা মাপে, Synthetic কৃত্রিম probe দিয়ে পরীক্ষা করে। দুটি একসঙ্গে ব্যবহার করলে বাইরের ও ভেতরের দৃশ্য দুটিই পাওয়া যায়। দুটি একই নয়।

93. B — দ্রুত burn ও ধীর burn আলাদা করে ধরা যায়, false positive কমে। Multi-window alert দ্রুত burn-এ তাড়াতাড়ি ও ধীর burn-এ ধৈর্য ধরে। শুধু একটি window-এ ভুল alert বেশি আসে।

94. D — ক্লাস্টারের সাম্প্রতিক events, যেমন scheduling ব্যর্থতা বা crash। Events বলে কী ঘটেছে, যেমন Pod schedule হয়নি কেন। এটি log নয়।

95. C — কোনো metric একেবারেই আসছে না — যেমন job বন্ধ — তা ধরতে। `absent()` metric অনুপস্থিত থাকলে 1 দেয়। Metric থাকলে alert করতে `up==0` ব্যবহার করুন।

96. B — Storage খরচ কমবে, কিন্তু 7 দিনের পুরোনো incident তদন্ত কঠিন হবে। Retention কমানো খরচ ও তদন্তের ট্রেড-অফ। Metrics হারায় না, শুধু log হারায়।

97. D — কোড হিসেবে data source সংজ্ঞায়িত করে পুনরুৎপাদনযোগ্য সেটআপ করা যায়। Provisioning YAML দিয়ে Grafana নতুন করে বসালেও একই data source পাওয়া যায়। এটি dashboard মুছে না।

98. B — Incident দেখা যায় না — blind spot তৈরি হয়। Platform down হলে monitoring অন্ধ হয়ে যায়। তাই platform-এর নিজের health আলাদাভাবে monitor করতে হয়।

99. A — Deploy সময়ের সঙ্গে error/latency বদলের সময় মিলিয়ে দেখা। Deploy-এর পরপরই error বাড়লে change-ই সন্দেহের প্রথম কারণ। Dashboard-এর রং প্রমাণ নয়।

100. D — Downstream সার্ভিস, DB query বা নেটওয়ার্ক অপেক্ষা — trace ও DB metrics দেখা। CPU স্বাভাবিক থাকলে সমস্যা অপেক্ষায় (waiting) — DB বা downstream-এ। CPU বাড়ানো সমাধান নয়।

101. C — Request অপেক্ষা করে, latency বাড়ে ও timeout/error বাড়ে। Pool পূর্ণ হলে নতুন request সংযোগের জন্য অপেক্ষা করে। CPU 0% হওয়া এর লক্ষণ নয়।

102. D — Scrape interval-এর চেয়ে ছোট হলে ডেটা মিস হয় ও ফলাফল অস্থির হয়। Range vector-এ অন্তত কয়েকটি sample থাকতে হয়। 1m range-এ 15s scrape-এ মাত্র 4টি sample থাকে।

103. A — ERROR। ERROR সবচেয়ে গুরুতর, তারপর WARN, INFO, DEBUG। DEBUG শুধু তদন্তের জন্য।

104. A — Volume, খরচ ও noise বাড়ে; গুরুত্বপূর্ণ log খুঁজে পাওয়া কঠিন হয়। Production-এ INFO/WARN/ERROR যথেষ্ট। DEBUG শুধু নির্দিষ্ট সময়ে চালু করুন।

105. A — কোনো সার্ভিস trace context forward করেনি বা header হারিয়েছে। Context হারালে span আলাদা trace হয়। Dashboard ভুল এর কারণ নয়।

106. D — Prometheus alert PromQL-এ, Grafana alert একাধিক data source-এ query করে। দুটির lifecycle ও পরিচালনা আলাদা। একসঙ্গে দুটি ব্যবহার করলে duplicate alert হতে পারে।

107. D — ব্যবহারকারী ও টিম জানে কী হচ্ছে, duplicate কাজ ও আতঙ্ক কমে। যোগাযোগ ছাড়া সবাই আলাদাভাবে একই কাজ করে। নিয়মিত আপডেট আস্থা বাড়ায়।

108. C — পরদিন পর্যালোচনা করে threshold/severity ঠিক করা; non-actionable alert-কে paging থেকে সরানো। প্রভাবহীন alert-এ রাতে জাগানো ভুল। Alert-এর উদ্দেশ্য পর্যালোচনা করুন।

109. A — ব্যবহারের জন্য উপলব্ধ memory। MemAvailable বলে কত memory ব্যবহার করা যায়। এটি মোট disk নয়।

110. D — 100% হলে লেখা বন্ধ হয়ে সার্ভিস ভেঙে পড়ে; আগেই ব্যবস্থা নেওয়ার সময় থাকে। 90%-এ alert দিলে cleanup বা disk বাড়ানোর সময় থাকে। 100%-এ অনেক দেরি হয়ে যায়।

111. B — নতুন series তৈরি হয় ও cardinality বাড়তে পারে। লেবেল বদলালে নতুন series তৈরি হয়। উদ্দেশ্য ছাড়া লেবেল বদলাবেন না।

112. D — সাধারণ HTTP/DB span পাওয়া যায়, কিন্তু ব্যবসায়িক যুক্তির span নিজে যোগ করতে হয়। Auto-instrumentation framework-এর কাজ ধরে, ব্যবসায়িক যুক্তি ধরে না। Manual span যোগ করুন।

113. D — Trace দেখে কোন span-এ সময় ব্যয় হচ্ছে তা খুঁজে বের করা। Metrics স্বাভাবিক ও log খালি থাকলে trace-ই পরবর্তী প্রমাণ। Restart করা অনুমান, তদন্ত নয়।

114. C — ব্যবহারকারীর অভিজ্ঞতা অনুযায়ী দুটিই আলাদা SLI হিসেবে মাপা উচিত। Availability ও latency আলাদা অভিজ্ঞতা মাপে। একটি দিয়ে অন্যটি বোঝা যায় না।

115. B — Node-এর CPU ও memory ব্যবহার। `kubectl top node` metrics-server থেকে Node-এর ব্যবহার দেখায়। Log দেখায় না।

116. D — কী মাপতে হবে তা আগে ঠিক করা, cardinality নিয়ন্ত্রণ, sampling ও retention tier ঠিক করা। খরচ নিয়ন্ত্রণ নকশা থেকে শুরু হয়, মুছে ফেলা থেকে নয়। সব ডেটা মুছে ফেললে তদন্ত অসম্ভব হয়।

117. B — বিভিন্ন উৎস থেকে log সংগ্রহ করে parse, filter ও Loki/Elasticsearch-এ পাঠানো। Alloy/Fluent Bit log pipeline-এর সংগ্রহকারী। Dashboard আঁকা এদের কাজ নয়।

118. C — সমস্যাটি নতুন change-এর সঙ্গে সম্পর্কিত হওয়ার শক্তিশালী প্রমাণ। Rollback-এ সমস্যা কমে গেলে change-ই কারণ হওয়ার সম্ভাবনা বেশি। এটি নেটওয়ার্কের প্রমাণ নয়।

119. C — এক জায়গায় metrics, logs ও traces একসঙ্গে দেখে সিদ্ধান্ত নেওয়া। Single pane of glass মানে আলাদা টুলে ঘুরতে হয় না। একটি panel নয়, একটি দৃশ্য।

120. D — সঠিক ডেটা, সঠিক সময়ে, সঠিক মানুষের কাছে — যাতে সিদ্ধান্ত ও পদক্ষেপ নেওয়া যায়। বেশি ডেটা নয়, কাজের ডেটাই observability-র উদ্দেশ্য। সব alert critical করলে আসল alert হারিয়ে যায়।

Case Studies — Analysis Guidance (10)

Case 1 — CPU Saturation: প্রমাণ: CPU 92%, p95 latency বেড়েছে, একটি Pod-এ চাপ কেন্দ্রীভূত। সম্ভাব্য কারণ: load balancing-এ সমস্যা বা ওই Pod-এর resource limit। তাৎক্ষণিক: Pod সংখ্যা বাড়ানো বা ট্রাফিক সরানো। দীর্ঘমেয়াদি: autoscaling, load balancing যাচাই, CPU limit পর্যালোচনা।

Case 2 — Memory Leak: প্রমাণ: memory ধীরে বাড়ছে, GC-র পরও কমছে না, heap-এ একই object বাড়ছে। এটি leak-এর শক্তিশালী প্রমাণ। সমাধান: heap profile দিয়ে leak-এর কোড খুঁজে ঠিক করা। তাৎক্ষণিক: restart ও memory alert ঠিক করা।

Case 3 — Disk Exhaustion: Alert আসেনি কারণ threshold 95% ও `for` সময় ঠিক ছিল না, এবং disk ধীরে ভরেছে। কারণ খুঁজতে: কোন ডেটা বেশি জায়গা নিচ্ছে তা দেখুন। প্রতিরোধ: 85% warning, 90% critical alert, retention কমানো, cleanup automation।

Case 4 — Latency Regression: Deploy-এর 5 মিনিট পর latency বেড়েছে — সময়ের মিল শক্তিশালী প্রমাণ। Trace-এ DB span ধীর, slow query-তে index নেই। Error বাড়েনি কারণ query সফল হচ্ছে, শুধু ধীর। Rollback দ্রুত, hotfix স্থায়ী — প্রভাব বেশি হলে rollback করুন।

Case 5 — Crash Loop: Root cause: DB সংযোগ ব্যর্থতা, অ্যাপ চালু হতে পারছে না। আগের সংস্করণে DB ঠিকানা বা credential ঠিক ছিল। Readiness probe থাকলে Pod traffic পেত না, কিন্তু crash থামত না — dependency check ও config যাচাই দরকার।

Case 6 — DB Connectivity: DB restart-এর পর pool পূর্ণ — DB down না pool/config সমস্যা, তা DB-তে সরাসরি সংযোগ করে যাচাই করুন। Retry storm DB-কে আরও চাপ দেয়। সমাধান: pool size, timeout, circuit breaker ও backoff ঠিক করা।

Case 7 — 5xx Spike: নিজের CPU/memory স্বাভাবিক, downstream 503 দিচ্ছে — প্রমাণ বলে সমস্যা downstream-এ। Spike কমেছে কারণ downstream সুস্থ হয়েছে বা retry কমেছে। প্রতিরোধ: timeout, circuit breaker, fallback ও downstream SLO monitoring।

Case 8 — Downstream Failure: Trace-এ gateway span-এই সময়, নিজের resource স্বাভাবিক, gateway-র status page-এ incident — সমস্যা নিজের সিস্টেমে নয়। ব্যবহারকারীকে জানান, checkout-এ fallback বা পরে চেষ্টা করার বিকল্প দিন। Timeout ও circuit breaker সেট করুন।

Case 9 — Alert Storm: একটিই root cause, কিন্তু grouping না থাকায় প্রতিটি সার্ভিস আলাদা alert পাঠিয়েছে। Grouping ও inhibition থাকলে একটি alert-এ সব তথ্য আসত। On-call-কে প্রথমে প্রভাব ও root cause দেখতে হবে, প্রতিটি alert আলাদা পড়তে হবে না।

Case 10 — Platform Outage: Platform-এর নিজের health check ছিল না, তাই blind spot তৈরি হয়েছে। বিকল্প: `kubectl logs`, সরাসরি API, ব্যবহারকারীর অভিযোগ। প্রতিরোধ: platform-এর আলাদা monitoring, redundancy, backup dashboard ও runbook।

Final Exam — Marking Guide

Part A (25): সব সার্ভিস healthy (10), target `up==1` (8), data source সংযুক্ত (7)। শুধু Compose চালু করলেই পূর্ণ নম্বর নয় — health প্রমাণ লাগবে।

Part B (20): Counter ও Histogram ঠিকভাবে instrumented (8), JSON log-এ সব field আছে (6), trace Tempo-তে দেখা যায় (6)। Trace_id log-এ না থাকলে নম্বর কাটা যাবে।

Part C (20): প্রতিটি PromQL/LogQL সঠিক ও ফলাফল ব্যাখ্যাসহ (প্রতিটি ~3)। ভুল query-তে আংশিক নম্বর, ব্যাখ্যা ছাড়া পূর্ণ নম্বর নয়।

Part D (20): চারটি Golden Signals panel উদ্দেশ্যসহ (10), দুটি alert rule `for`/severity/runbook-সহ (7), test notification যাচাই (3)।

Part E (15): Root cause প্রমাণসহ (8), timeline ও প্রভাব (4), প্রতিরোধমূলক পদক্ষেপ (3)। অনুমানকে প্রমাণ বলে চালালে নম্বর কাটা যাবে।

সাধারণ নিয়ম: প্রতিটি ধাপে প্রমাণ (query ফলাফল, log লাইন, trace ID) থাকতে হবে। কপি করা config-এর ব্যাখ্যা না থাকলে নম্বর মিলবে না।

Capstone Grading Rubric

Criteria	Weight	Excellent (90–100%)	Good (70–89%)	Needs Work (below 70%)
Observability stack deployment	20%	সব সার্ভিস healthy, HA ও retention নকশাসহ	সব সার্ভিস চলছে, কিছু নকশা বাদ	সার্ভিস চালু হয় না বা target down
Instrumentation (metrics, logs, traces)	20%	তিনটি স্তম্ভ যুক্ত, trace_id সর্বত্র	দুটি স্তম্ভ ঠিক, একটি আংশিক	শুধু একটি স্তম্ভ বা ভুল instrumentation
Dashboards & PromQL/ LogQL	20%	Golden Signals dashboard, সঠিক query, ব্যাখ্যাসহ	Dashboard আছে, কিছু query ভুল	Dashboard নেই বা query কাজ করে না
Alerting & SLO	15%	Actionable alert, SLO ও burn rate-সহ	Alert আছে, SLO আংশিক	Alert নেই বা non-actionable
Incident investigation	15%	প্রমাণ দিয়ে root cause, timeline-সহ	Root cause ঠিক, প্রমাণ আংশিক	অনুমানভিত্তিক, প্রমাণ নেই
Documentation & postmortem	10%	স্পষ্ট runbook ও blameless postmortem	ডকুমেন্ট আছে, কিছু অংশ বাদ	ডকুমেন্ট নেই বা অস্পষ্ট

Pass standard: মোট 60% বা তার বেশি, এবং Incident Investigation-এ অন্তত 60%। Distinction-এর জন্য মোট 80% বা তার বেশি এবং প্রতিটি criteria-তে অন্তত 70%।